Pharmacy Inventory & Prescription Management System

Database Systems Lab (BCSE302P)

PROJECT REPORT

Fall Semester 2026-2027

Submitted by

ABHIRUP BANIK (24BCE0311)
SARVASWA BHANTI (24BCE0333)
AMOGH SHARMA (24BCE2126)
SAMI FARHAN SYED (24BCE2744)

in partial fulfillment for the award of the degree of

B. Tech

in

Computer Science and Engineering
Core



Vellore-632014, Tamil Nadu, India

School of Computer Science and Engineering

October, 2026

CONTENTS

Pharmacy Inventory & Prescription Management System

1. **Abstract**
2. **Introduction**
3. **Background Study**
4. **Methodology**
    - **4.1 Explanation of the Working Model and Dataset**
    - **4.2 Proposed Working Model and ER Diagram**
    - **4.3 Modules and Sub-modules**
5. **Results and Discussion**
6. **Conclusion**
7. **References**

1. ABSTRACT

A pharmacy has to do two things correctly every time a medicine is handed over: it must take the right physical stock from the right batch without ever losing count, and it must make sure the medicine does not interact dangerously with the other medicines on the same prescription. These two requirements pull in different directions. Stock and prescriptions need strict consistency, row-level locking and an audit trail that cannot be altered, while drug interaction knowledge has no fixed shape, arrives from outside sources and is read far more often than it is written.

This project builds a working pharmacy inventory and prescription management system around that observation, using polyglot persistence. **PostgreSQL 17** is the transactional core: medicines, batches, prescriptions and an append-only dispensing log, with every dispense executed as one short ACID transaction that locks the prescription line and the batch, picks stock First-Expiry-First-Out (FEFO) through a partial B-tree index, and writes the stock change and the audit row together. **MongoDB 9** is the safety layer: a drug_interactions collection whose documents carry different fields for different drug pairs, protected by a $jsonSchema validator and a unique compound index. Every dispense checks MongoDB before and inside the transaction; if MongoDB is unreachable, the dispense is refused and rolled back.

The system is delivered as a Node.js and Express API with a React front end. Measured on a seeded database of 60,181 batches, the FEFO query runs in under 0.2 ms with the index against 7 to 13 ms without it, two pharmacists competing for the last unit are serialised correctly by the row lock, and 24 of 24 end-to-end checks pass.

2. INTRODUCTION

Dispensing is the point at which a pharmacy's records meet the physical world. A pharmacist reads a prescription, takes tablets from a shelf and hands them over. Behind that simple act sit several rules that a database has to enforce without exception: stock must never go negative or be counted twice, the oldest stock that is still in date should leave first so that nothing expires on the shelf, a prescription must not be dispensed beyond the quantity written on it, and every movement of stock must leave a record that shows who did what and when.

A second, quite different concern runs alongside. Many medicines are dangerous in combination: aspirin with warfarin raises the risk of bleeding, sildenafil with a nitrate can cause a fatal fall in blood pressure, clarithromycin with simvastatin can cause muscle breakdown. Whether a combination matters, and what to do about it, is clinical knowledge that differs from pair to pair. Some interactions depend on a dose threshold, some on the patient's age, some need a gap of several hours between doses, and some are combinations that are intended. Forcing this knowledge into fixed columns produces wide, mostly empty tables or an attribute table that needs joins for every lookup.

The project treats these two concerns as two kinds of data and gives each the database that suits it, an approach known as polyglot persistence [1], [2]. The relational engine handles everything that must be exact and transactional; the document store handles knowledge whose shape varies. The interesting engineering lies in where the two meet: a dispense must never complete without the safety check, yet the transaction must stay short enough that pharmacists working in parallel are not blocked by each other.

2.1 Objectives

1) Model medicines, batches, prescriptions and users relationally, with keys and CHECK constraints that make invalid states impossible to store.

2) Dispense in **one ACID transaction**: lock the prescription line and the batch, choose the batch by FEFO, update stock and write the audit row together, or roll everything back.

3) Keep a **tamper-proof audit log** that the application itself cannot rewrite.

4) Store drug interactions as **flexible documents** in MongoDB and check every pair of medicines on a prescription before dispensing; refuse to dispense if the check cannot be made.

5) Make the behaviour **visible**: show the transaction steps, the concurrency race, the outage, the query plan and the audit guard live, and measure them.

2.2 Scope

The system covers prescription entry, dispensing, stock receipt, inventory and expiry views, the audit log with a weekly report, and the interaction knowledge base. Billing, supplier ordering and user authentication are outside the scope; the acting pharmacist is chosen from a list so that every audit row can name a person.

3. BACKGROUND STUDY

Relational model and transactions. Codd's relational model [3] is the basis for representing stock and prescriptions as normalised relations linked by keys. The ACID properties and the theory of locking used here are set out by Gray and Reuter [4]. Berenson et al. [5] showed that the ANSI isolation levels leave room for anomalies such as lost updates; the standard remedy, which this project uses, is to lock the rows being changed explicitly with SELECT ... FOR UPDATE, as documented for PostgreSQL [6]. Silberschatz, Korth and Sudarshan [7] describe B-tree indexing, which the FEFO query relies on, and PostgreSQL's partial indexes [8] allow the index to contain only batches that still hold stock.

Polyglot persistence. Stonebraker and Cetintemel [9] argued that a single database architecture cannot serve every workload well. Sadalage and Fowler [1] named the practice of using several data stores, each for the data it suits, polyglot persistence, and Kleppmann [2] discusses the trade-offs, in particular the difficulty of keeping several systems consistent. This project limits that difficulty by writing to only one system during a dispense: MongoDB is read, never written, inside the dispensing path, so no distributed commit is needed.

Document stores and schema flexibility. Hecht and Jablonski [10] survey NoSQL stores and identify document stores as suitable for records whose attributes vary. MongoDB adds optional schema validation with $jsonSchema [11], which lets a collection require a minimum set of fields while accepting any others; this is the "flexibility with a floor" used for the interaction documents.

Drug interaction data. Public knowledge bases such as DrugBank [12] record drug-drug interactions with a description, severity and management advice, and the shape of each record varies with what is known about the pair. The 30 interaction documents seeded in this project follow that pattern.

Stock rotation. The World Health Organization's good storage and distribution practices for medical products [13] recommend that stock be issued so that the earliest expiry leaves first (FEFO), and that expired stock be kept apart and never issued. Both rules are enforced by the FEFO query in this project.

Table 1: Summary of related work and how it is used

Work	Contribution	Used in this project
Codd [3]; Silberschatz et al. [7]	Relational model, normalisation, B-tree indexes	Six-table schema in 3NF; FEFO index
Gray and Reuter [4]; Berenson et al. [5]	ACID transactions; isolation anomalies	One dispense = one transaction with row locks
PostgreSQL documentation [6], [8]	SELECT ... FOR UPDATE; partial indexes	Lock order; idx_batches_fefo WHERE quantity > 0
Stonebraker and Cetintemel [9]; Sadalage and Fowler [1]; Kleppmann [2]	One size does not fit all; polyglot persistence	PostgreSQL for stock, MongoDB for interactions
Hecht and Jablonski [10]; MongoDB documentation [11]	Document stores; schema validation	Variable-field documents with a validator
DrugBank [12]; WHO [13]	Interaction records; FEFO and expiry rules	Seed knowledge; FEFO and expired-stock handling

4. METHODOLOGY

4.1 Explanation of the Working Model and Dataset

The system has three layers. The pharmacist works in a React application in the browser. The browser calls an Express API over HTTP with JSON, and only the API talks to the databases, so every rule is enforced on the server or inside the database and never in the browser. The API keeps two PostgreSQL connection pools, one per database role, and one MongoDB client with a 1.5 second timeout.

Table 2: Technology stack

Layer	Technology	Role
Transactional database	PostgreSQL 17.11	Stock, prescriptions, audit log; ACID transactions, row locks, constraints, indexes, trigger, roles
Safety database	MongoDB 9.0.2	drug_interactions documents; $jsonSchema validator; unique compound index
API server	Node.js 22, Express 4	21 REST routes; the two-phase dispense; serves the built front end
Drivers	node-postgres 8, MongoDB driver 6	Explicit BEGIN / COMMIT / ROLLBACK; bounded server-selection timeout
Front end	React 18, Vite 6	Six screens; works on laptop and mobile browsers
Tooling	setup.js, smoke.js, export.js	Rebuild both databases; 24 end-to-end checks; export data to CSV and JSON

The dispensing workflow is split into two phases. In the preview phase the pharmacist picks a line of a prescription; the API reads the batches of that medicine in expiry order, plans which batches the quantity will come from, and asks MongoDB for every interaction between the generic names on the prescription. Nothing is locked or written, so the pharmacist can read a warning for as long as needed. In the confirm phase the API opens one transaction and:

1) locks the prescription line with SELECT ... FOR UPDATE and refuses a quantity above what remains prescribed;

2) locks the earliest unexpired batch with stock, and the next one if the first is too small;

3) checks MongoDB again; a contraindicated pair, an unacknowledged high or moderate warning, or an unreachable MongoDB ends in ROLLBACK;

4) reduces each batch with UPDATE and writes one DISPENSE row per batch to dispensing_events;

5) commits, which releases the locks and makes the stock change and the audit rows visible together.

Because every transaction locks the prescription line first and then batches in expiry order, two transactions can never wait for each other in a cycle, so deadlock cannot occur. Receiving stock follows the same pattern: the new batch and a RECEIPT row are inserted in one transaction.

Figure 1 summarises, for every action the pharmacist can take, which tables and collections are read, written or locked, and why.

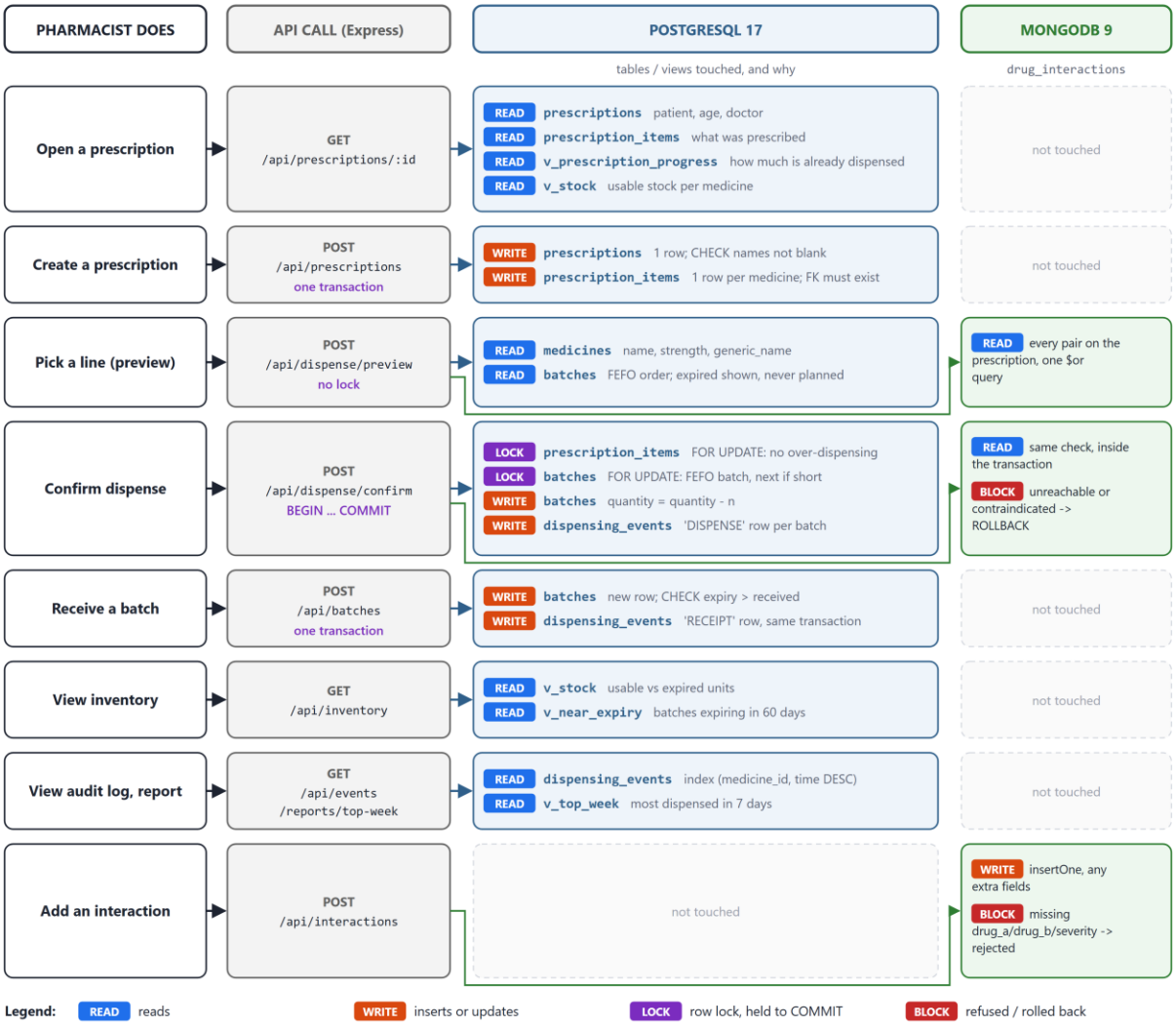


Figure 1: Data flow: for each action, the API call, the PostgreSQL tables touched and the MongoDB use

Dataset. The databases are built from scratch by setup.js in about two seconds, from SQL files and a seed. Dates in the seed are relative to the current date, so the data never goes stale. The PostgreSQL side holds 44 real medicines with generic names, 181 live batches with staggered expiries (including expired batches for some medicines), and 60,000 emptied historical batches so that the indexes have realistic work to do. Ten prescriptions are seeded to cover every case: a clean prescription, high, moderate and low interactions, and a contraindicated pair. 3,000 historical dispenses are followed by receipt events chosen so that the audit log sums exactly to the stock on hand.

Table 3: Seed dataset

Data	Rows / documents	Purpose
medicines	44	Products with generic name, form, strength, pack, manufacturer
batches	181 live, 60,181 total	Staggered expiries; expired lots; history that makes the FEFO index matter
prescriptions / items	10 / 24	Every interaction severity and a contraindicated pair
users	4	The team as pharmacists and admin
dispensing_events	3,181	History of dispenses and receipts that reconciles with stock
drug_interactions (MongoDB)	30 documents, 18 field names	Real interaction pairs with varying fields

4.2 Proposed Working Model of the Project and ER Diagram

Figure 2 shows the complete working model. The front end has six screens; the API layer has three modules; the data layer holds six tables, four views, the indexes, an audit trigger and two roles in PostgreSQL, and one validated collection in MongoDB.

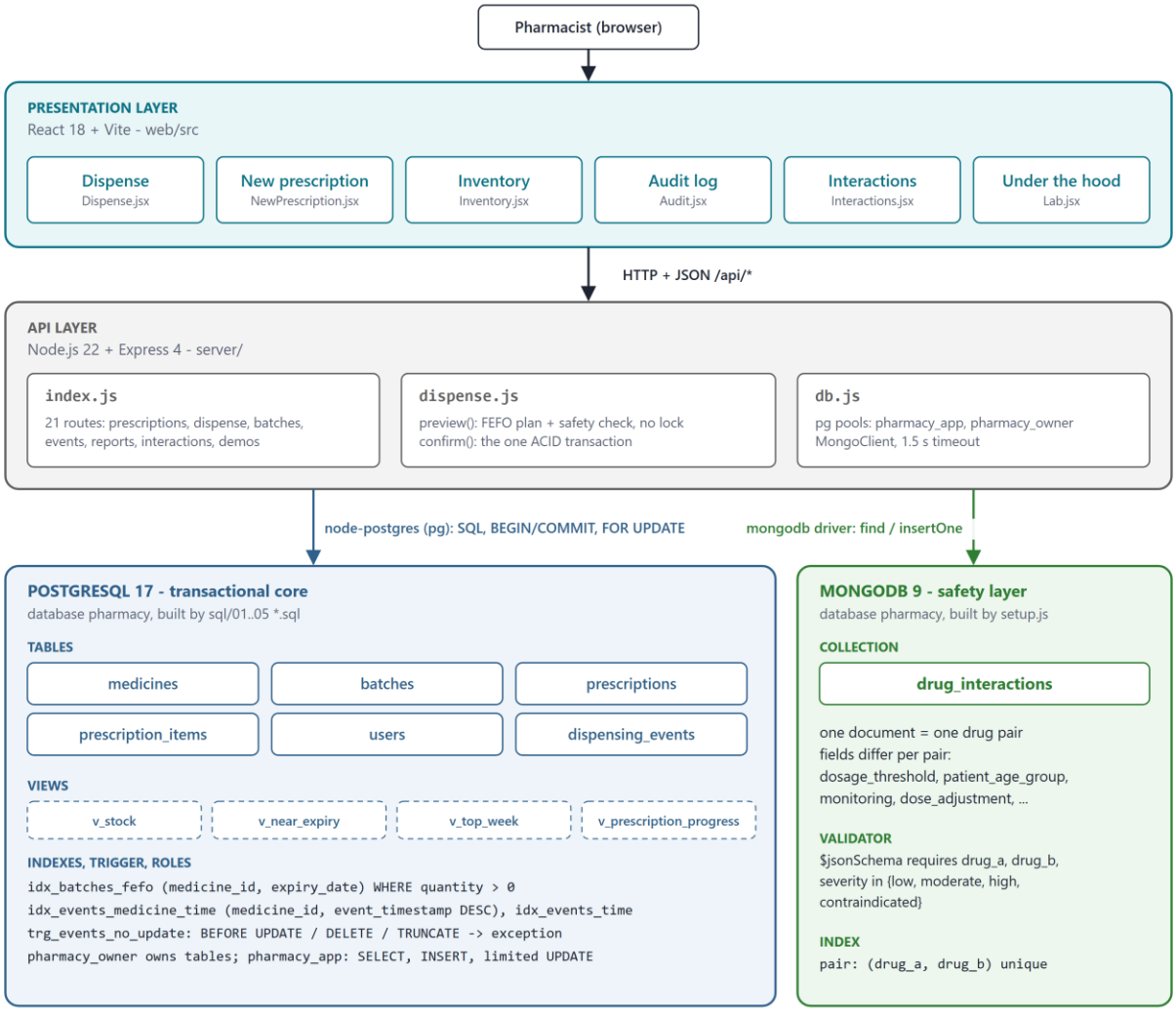


Figure 2: Proposed working model: three layers, the tech stack and the objects in each database

The ER diagram in Figure 3 shows the five entities and their relationships. A prescription contains many medicines and a medicine appears on many prescriptions; the relationship carries the quantity prescribed and becomes the prescription_items table. A medicine has many batches. Every dispensing event is performed by one user, takes stock from one batch, is of one medicine and may cite one prescription.

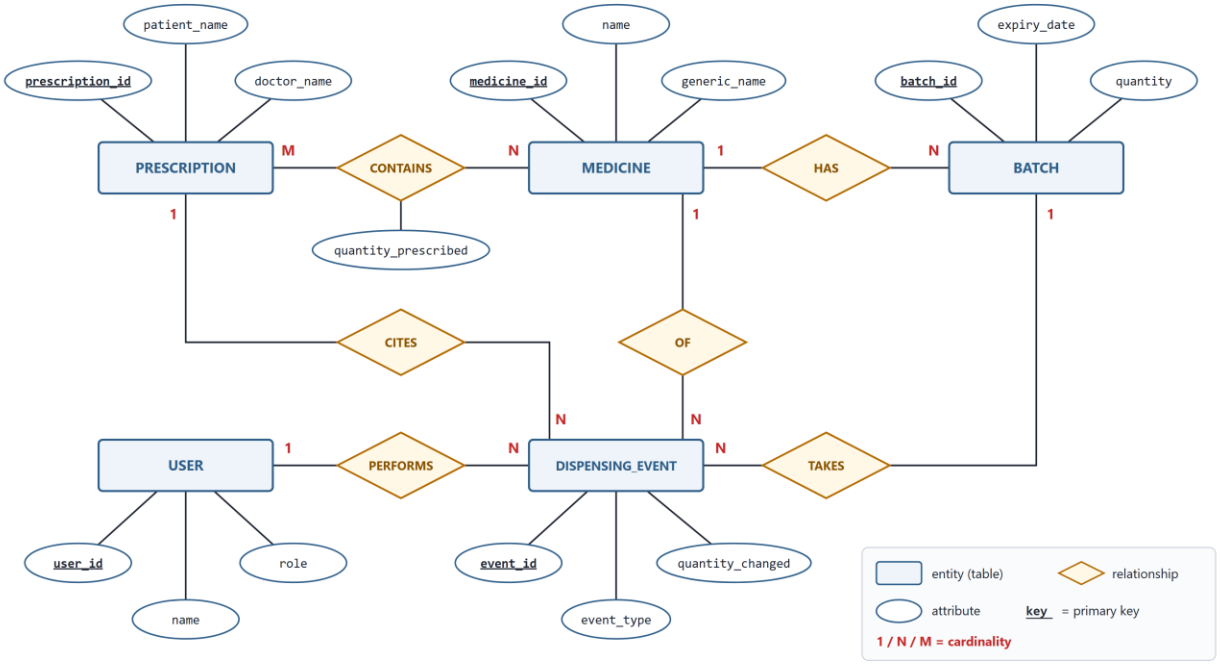


Figure 3: ER diagram of the system

Figure 4 maps the ER diagram to the relational schema actually created, with every column, type and constraint. The schema is in third normal form: every non-key attribute depends on the whole key and nothing but the key. The dashed green line marks the single value shared by the two databases: medicines.generic_name is the drug_a / drug_b key in MongoDB.

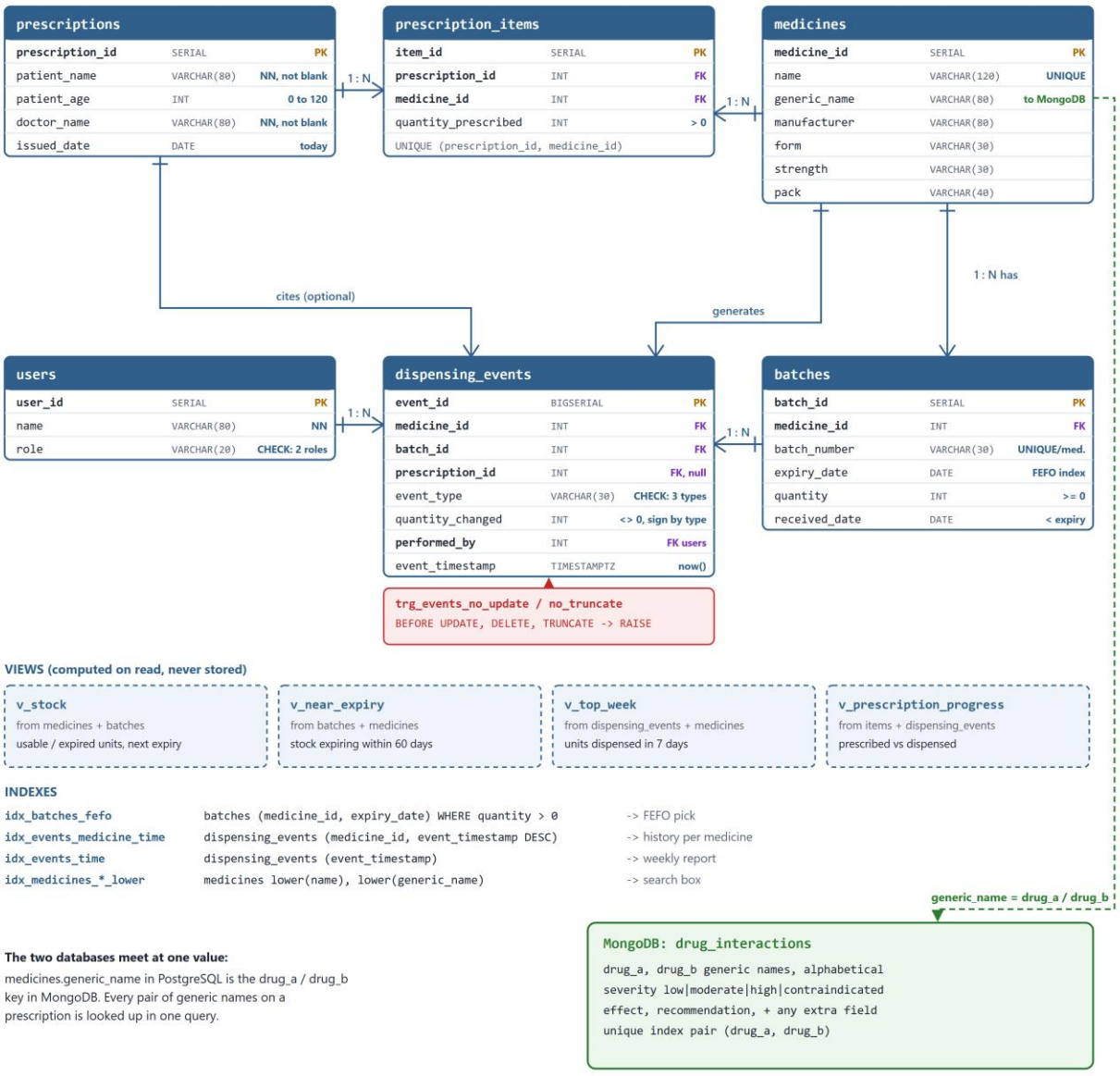


Figure 4: Relational schema with keys, constraints, trigger, views, indexes and the MongoDB collection

Table 4: What each table stores

Table	Stores	Written by / read by
medicines	One row per product; generic_name links to MongoDB	Seeded; read by search, preview and reports

Table	Stores	Written by / read by
batches	One row per physical lot: number, expiry, quantity, received date; the source of truth for stock	Inserted on receipt; reduced on dispense under a row lock
prescriptions	Patient, age, doctor, date issued	Inserted by New prescription; read on open
prescription_items	One row per medicine on a prescription with quantity prescribed	Inserted with the prescription; locked on confirm
users	Pharmacists and admin	Seeded; referenced by every audit row
dispensing_events	Append-only DISPENSE / RECEIPT / ADJUSTMENT history	Inserted with each stock change; never updated
drug_interactions	One document per drug pair, fields vary	Read on every preview and confirm; written rarely

4.3 Modules and Sub-modules

The system is organised into six modules on a common platform layer, shown in Figure 5.

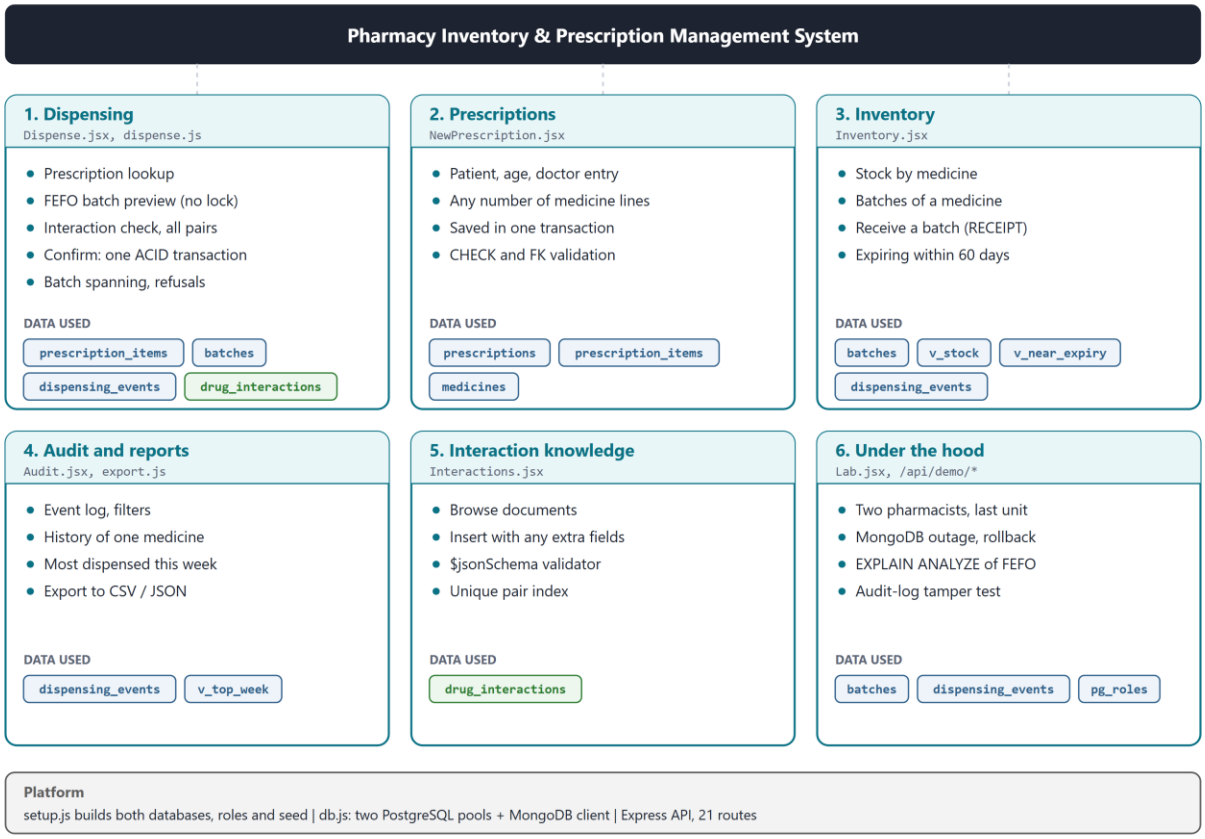


Figure 5: Modules, sub-modules and the data each one uses

1. Dispensing. The core module. Prescription lookup lists each line with the quantity prescribed, already dispensed and in stock (view v_prescription_progress). The FEFO preview shows the batches in expiry order with the chosen batch starred and expired lots struck through. The interaction check lists every warning for the whole prescription with its recommendation, mechanism and monitoring fields. The confirm sub-module runs the transaction of Figure 6 and shows its steps with timings.

2. Prescriptions. A form for patient, age, doctor and any number of medicine lines. One transaction inserts the prescription and its items; CHECK constraints reject a blank name or an impossible age, and the foreign key rejects an unknown medicine, so nothing is ever half-written.

3. Inventory. Stock by medicine from the v_stock view, with usable and expired units counted apart; the batches of a selected medicine; a form to receive a new batch, which writes the batch and a RECEIPT row together; and the list of stock expiring within 60 days from v_near_expiry.

4. Audit and reports. The dispensing_events log, filtered by medicine and event type; filtering by medicine uses the index on (medicine_id, event_timestamp DESC). The most dispensed medicines of the week come from v_top_week. All data can be exported to CSV and JSON.

5. Interaction knowledge. Browses the MongoDB documents and inserts new ones with any extra fields. The validator rejects a document without drug_a, drug_b or a valid severity, and the unique pair index rejects duplicates.

6. Under the hood. Four live demonstrations of the database concepts: two pharmacists racing for the last unit, a MongoDB outage during a dispense, EXPLAIN ANALYZE of the FEFO query with and without the index, and an attempt to rewrite the audit log as both database roles.

Figure 6 shows the dispense transaction in detail, including the three ways it ends in ROLLBACK.

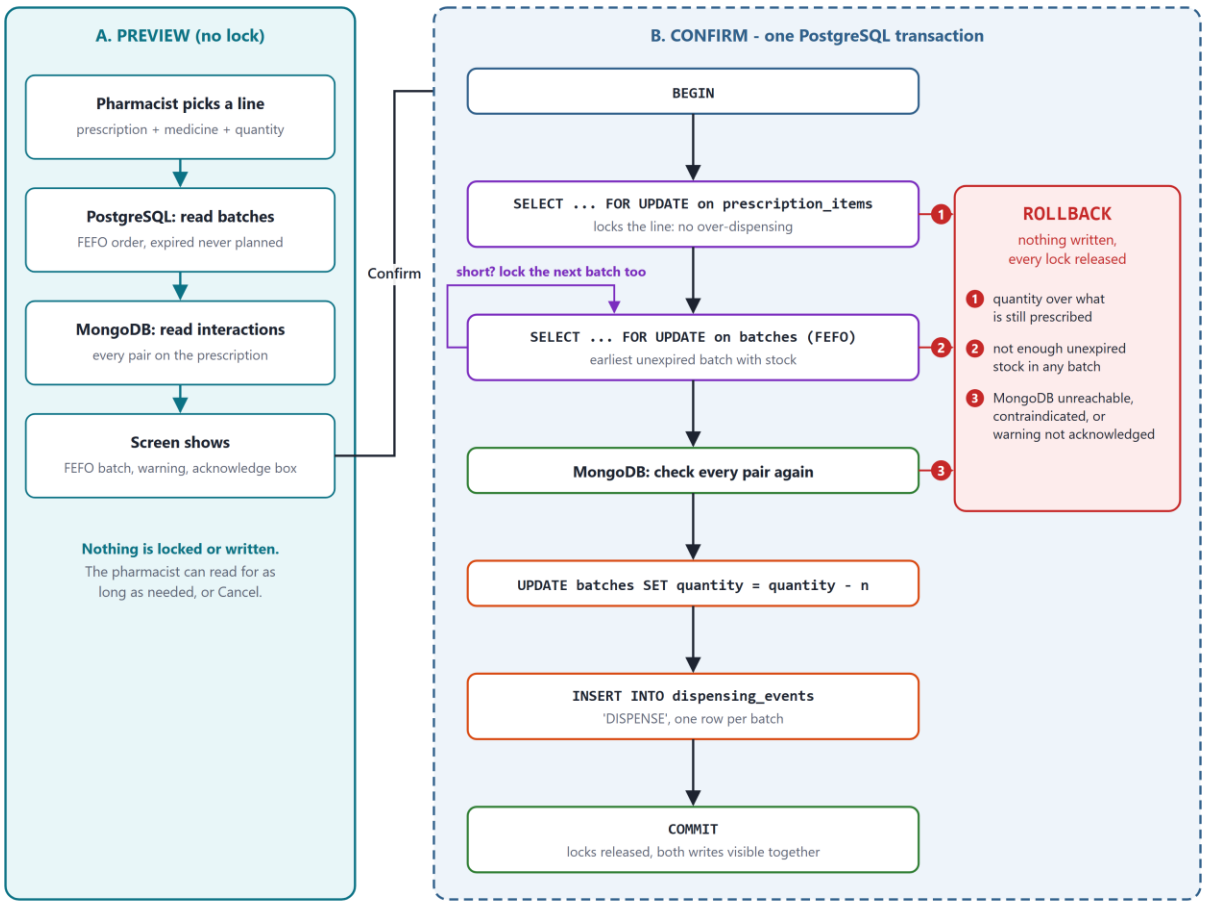


Figure 6: The dispense transaction: lock-free preview, then one short confirm transaction

5. RESULTS AND DISCUSSION

Every feature was exercised on the seeded database, both by hand through the screens shown below and automatically by smoke.js, which runs 24 end-to-end checks against the running API. All 24 pass. The measured behaviour of the four database concepts is summarised in Table 5.

Table 5: Measured results

Test	Result	What it shows
FEFO query, EXPLAIN ANALYZE (5 runs)	0.04 to 0.16 ms with the partial index; 7.2 to 12.9 ms with index scans off. Index scan reads 3 buffers; the sequential scan reads 443 and filters out 60,178 rows	Only 180 of 60,182 batches hold stock at the time of the run, and the partial index stores only those
Two pharmacists, last unit	B waits from about 255 ms to about 2,025 ms and takes the lock within 1 ms of A's COMMIT, then takes the next batch	The row lock serialises the two; the unit is sold once
MongoDB unreachable	HTTP 503 and ROLLBACK after the 1.5 s timeout; the same dispense commits once MongoDB is back	No dispense without the safety check
Audit log rewrite	pharmacy_app: refused, error 42501 (no privilege). pharmacy_owner: refused by the trigger	The audit log is append-only by the database itself
Unsafe dispenses	Over-prescription, contraindicated pair and unacknowledged warning all return 409 and ROLLBACK	Safety rules hold even if the screen is bypassed
Audit reconciliation	Sum of quantity_changed per batch equals batch quantity, 0 mismatches	History and stock agree
End-to-end checks	smoke.js: 24 of 24 pass	All rules hold together

Discussion. The most important design decision was to separate the preview from the confirm. An earlier design held the batch lock while the warning was on screen; that would have blocked every other pharmacist working on the same medicine for as long as one person read. With the split, locks are held only for the few tens of milliseconds the transaction takes, and the race

demonstration shows that correctness is not lost: the second transaction waits, re-reads the row after the first commits and moves on to the next batch.

The partial index is the reason the FEFO query stays fast as history grows. Emptied batches never enter the index, so its size depends on live stock rather than on the age of the system. Keeping MongoDB out of the write path removed the need for any cross-database commit: a dispense writes only to PostgreSQL, and MongoDB is a read that must succeed for the transaction to commit. The cost of polyglot persistence is a second system that must be available; the design chooses safety and refuses to dispense when it is not.

Limitations. User authentication is not implemented, the interaction data is a curated seed rather than a licensed source, and the system runs on one machine. None of these affect the database design, which is the subject of the project.

5.1 Screenshots of All Modules

The screenshots below follow a complete session, module by module, on the seeded data.

Module 1: Dispensing

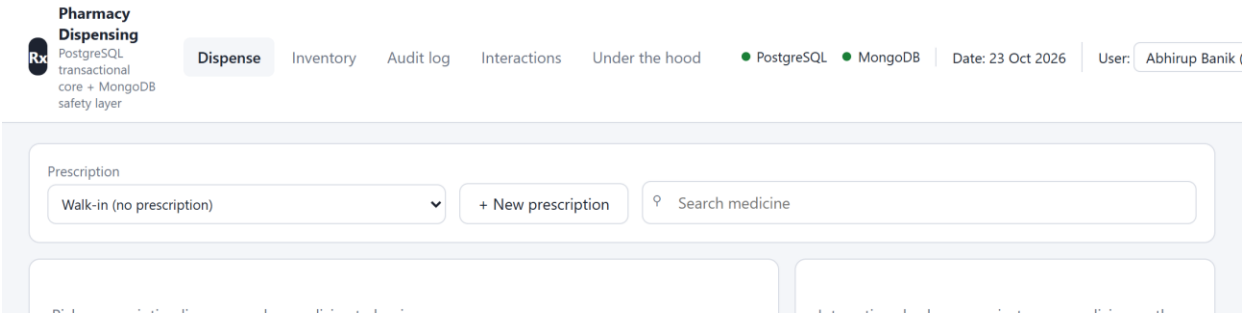


Figure 7: Dispense screen at start; both databases reachable

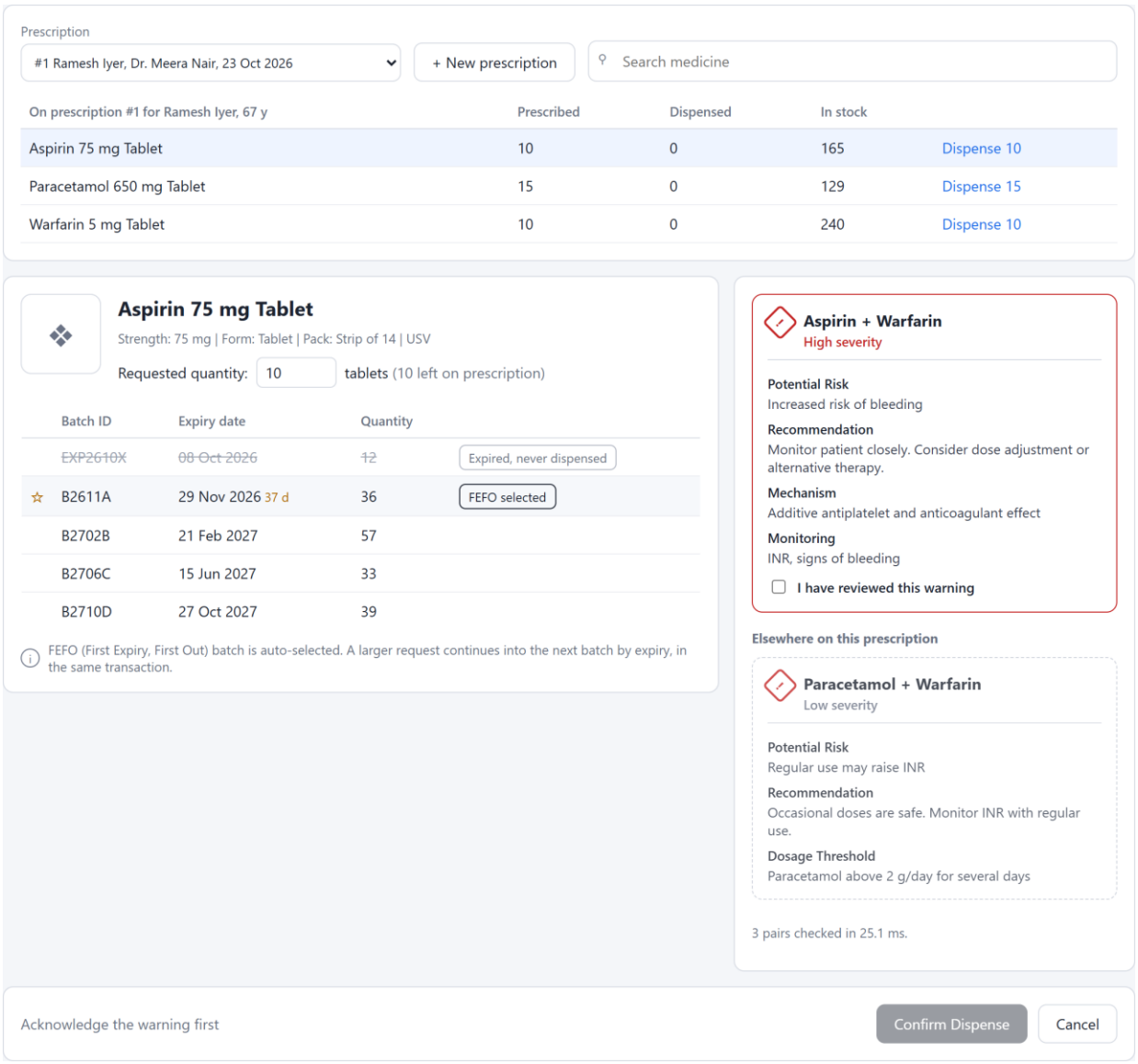


Figure 8: Preview: FEFO batch starred, expired batch struck through, warnings for every pair

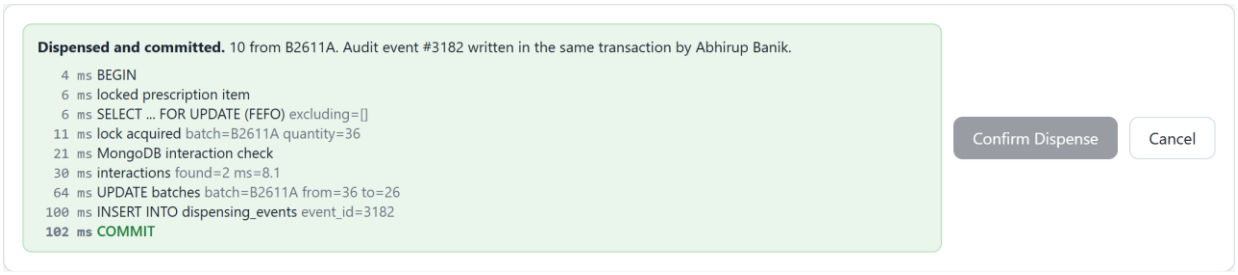


Figure 9: Confirm: the transaction steps from BEGIN to COMMIT with timings

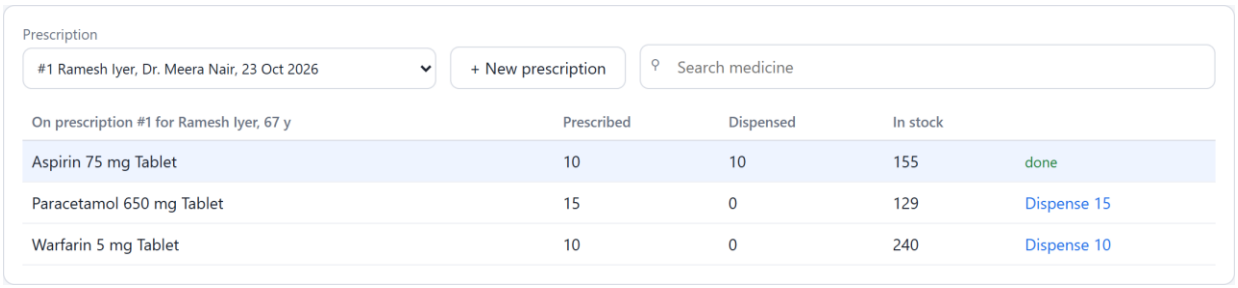


Figure 10: The prescription after the dispense; stock reduced

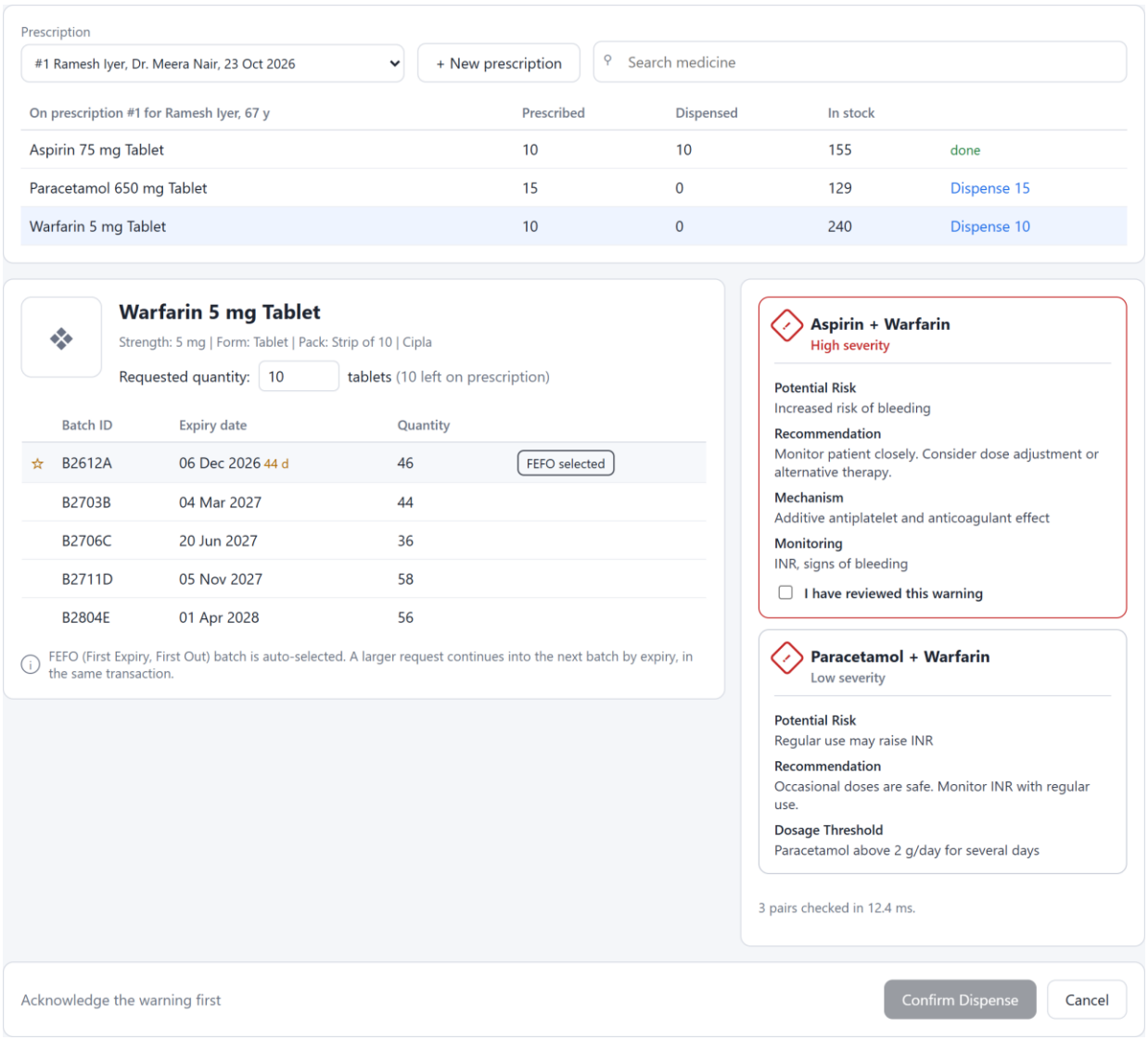


Figure 11: An unacknowledged high-severity warning blocks Confirm

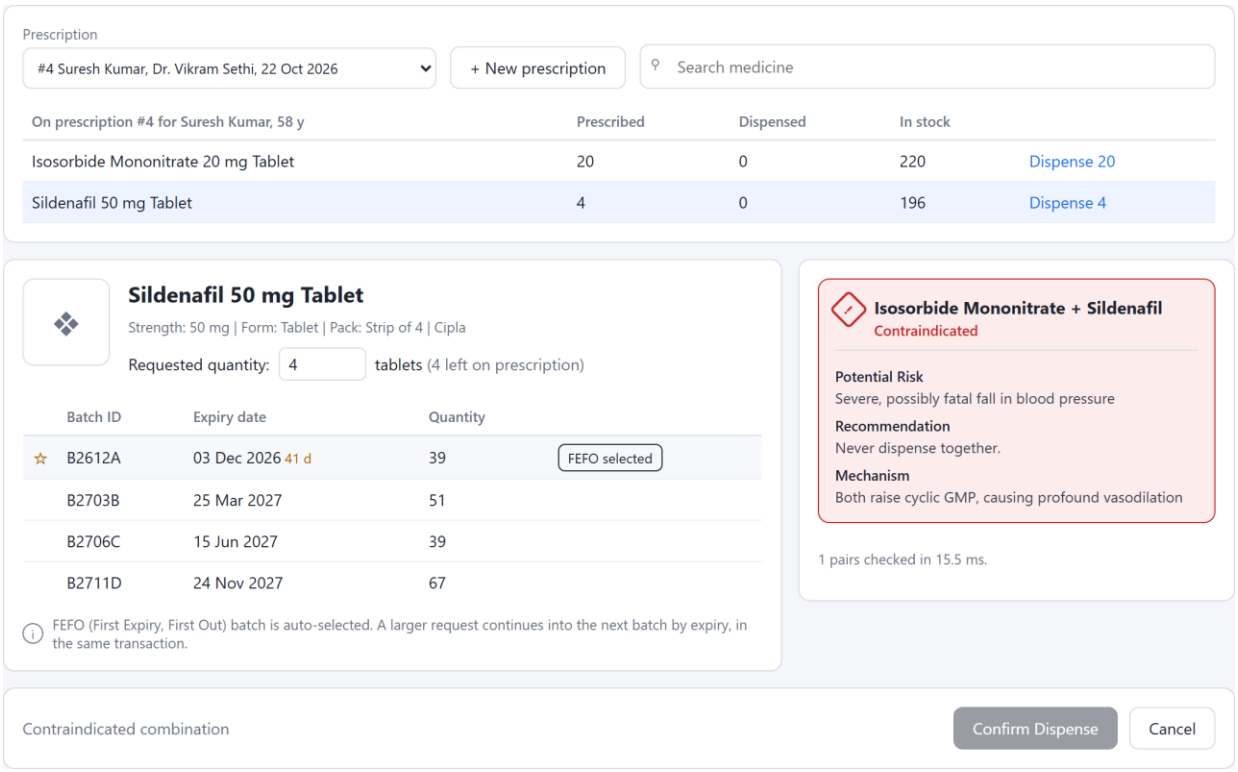


Figure 12: A contraindicated combination cannot be dispensed

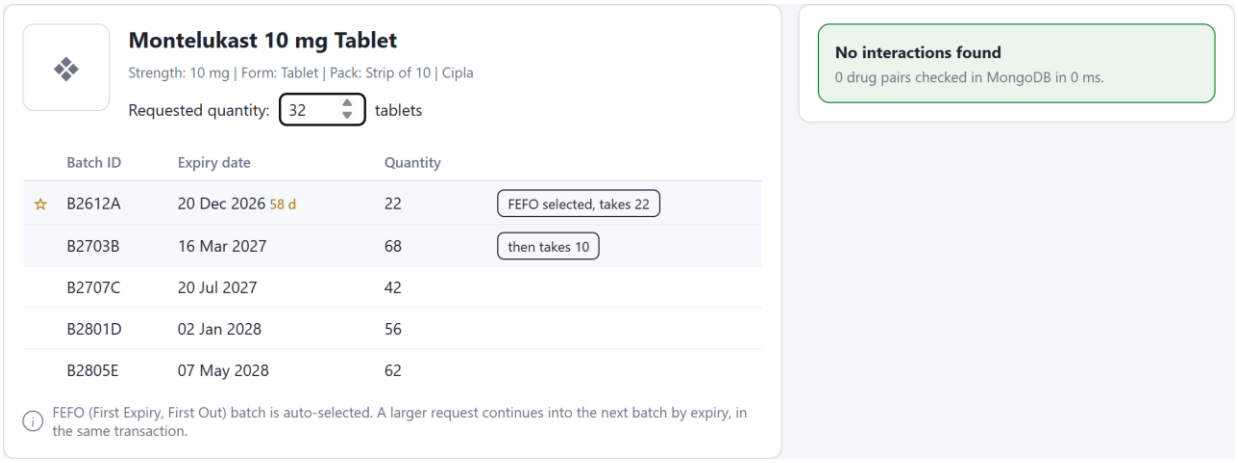


Figure 13: A request larger than one batch: the plan spans two batches

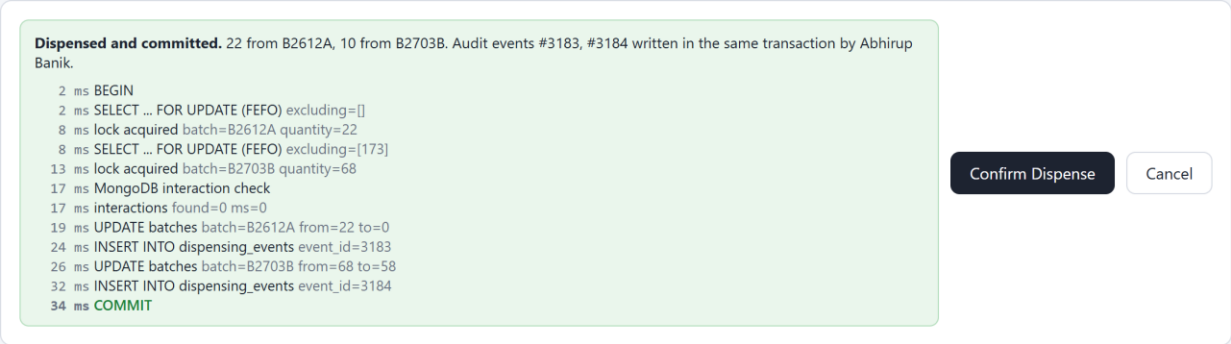


Figure 14: Committed across two batches, one audit row per batch

Module 2: Prescriptions

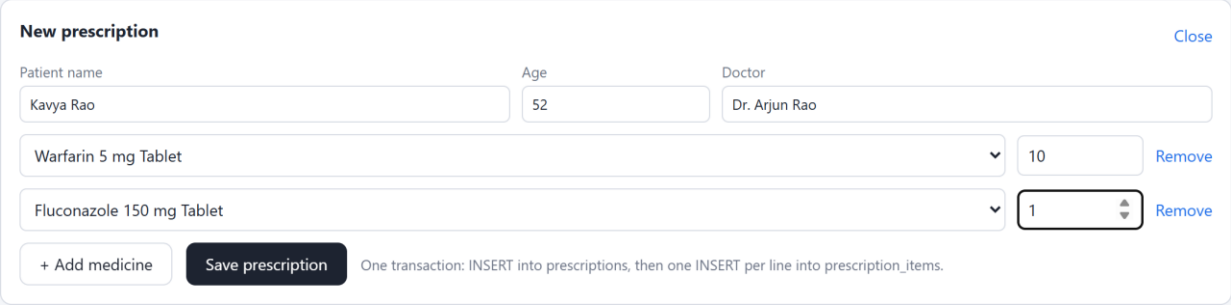


Figure 15: New prescription form with two medicine lines

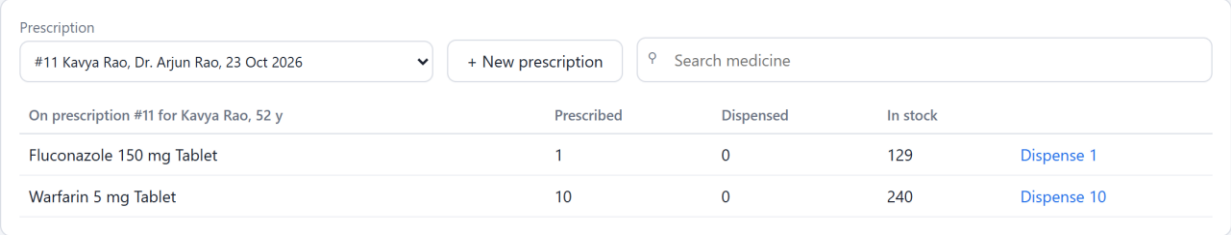


Figure 16: Prescription saved in one transaction and opened

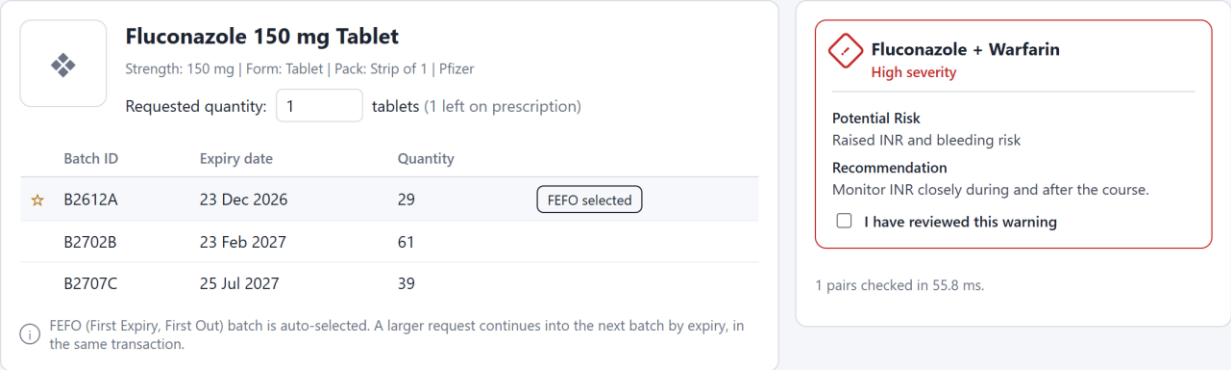


Figure 17: The new prescription raises its interaction warning at once

Module 3: Inventory



Figure 18: Receiving a new batch

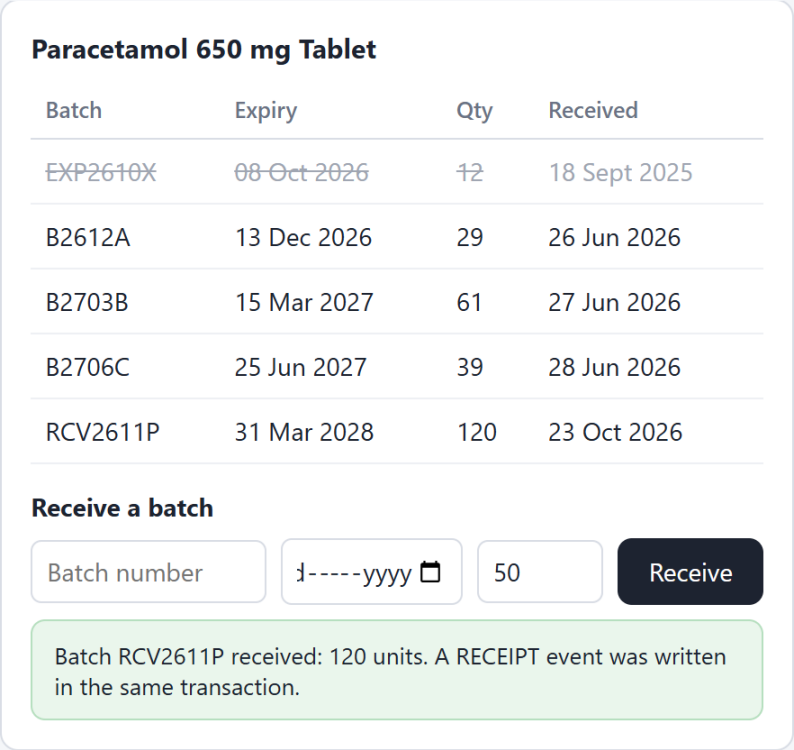


Figure 19: Batch received; RECEIPT written in the same transaction



Stock by medicine

Filter

From the view v_stock. Expired units are counted apart and never offered for dispensing.

Medicine	Usable	Expired	Next expiry	Batches
Allopurinol 100 mg Tablet	196	0	28 Dec 2026	4
Amiodarone 200 mg Tablet	180	0	07 Dec 2026	4
Amlodipine 5 mg Tablet	126	12	26 Nov 2026	4
Amoxicillin 500 mg Capsule	166	0	13 Dec 2026	4
Aspirin 75 mg Tablet	155	12	29 Nov 2026	5
Atorvastatin 10 mg Tablet	180	0	22 Dec 2026	4
Azathioprine 50 mg Tablet	225	0	25 Nov 2026	5
Azithromycin 500 mg Tablet	147	0	25 Dec 2026	3
Calcium Carbonate 500 mg Tablet	182	0	09 Dec 2026	4
Carbamazepine 200 mg Tablet	225	0	29 Nov 2026	5
Cetirizine 10 mg Tablet	126	0	06 Dec 2026	3
Ciprofloxacin 500 mg Tablet	135	0	17 Dec 2026	3
Clarithromycin 500 mg Tablet	215	0	29 Dec 2026	5
Clopidogrel 75 mg Tablet	138	0	24 Nov 2026	3
Digoxin 0.25 mg Tablet	270	0	14 Dec 2026	5
Fluconazole 150 mg Tablet	129	0	23 Dec 2026	3
Fluoxetine 20 mg Capsule	220	0	12 Dec 2026	5
Furosemide 40 mg Tablet	141	0	21 Dec 2026	3
Glimepiride 2 mg Tablet	198	0	30 Dec 2026	4
Hydroxychloroquine 200 mg Tablet	250	0	18 Dec 2026	5
Ibuprofen 400 mg Tablet	178	12	20 Dec 2026	5
Isosorbide Mononitrate 20 mg Tablet	220	0	10 Dec 2026	5

Figure 20: Stock by medicine from v_stock: usable and expired units

Expiring within 60 days

Batch	Medicine	Expiry	Qty
~~EXP2610X~~	~~Ibuprofen 400 mg Tablet~~	~~expired~~	12
~~EXP2610X~~	~~Amlodipine 5 mg Tablet~~	~~expired~~	12
~~EXP2610X~~	~~Aspirin 75 mg Tablet~~	~~expired~~	12
~~EXP2610X~~	~~Paracetamol 650 mg Tablet~~	~~expired~~	12
B2611A	Rifampicin 450 mg Capsule	30 d	30
B2611A	Methotrexate 2.5 mg Tablet	31 d	49
B2611A	Clopidogrel 75 mg Tablet	32 d	38
B2611A	Azathioprine 50 mg Tablet	33 d	37
B2611A	Amlodipine 5 mg Tablet	34 d	26
B2611A	Metformin 500 mg Tablet	35 d	25
B2611A	Lisinopril 10 mg Tablet	36 d	44
B2611A	Carbamazepine 200 mg Tablet	37 d	43
B2611A	Aspirin 75 mg Tablet	37 d	26
B2611A	Trimethoprim 100 mg Tablet	38 d	32
B2612A	Omeprazole 20 mg Capsule	39 d	21
B2612A	Levothyroxine 50 mcg Tablet	40 d	20
B2612A	Sildenafil 50 mg Tablet	41 d	39
B2612A	Prednisolone 10 mg Tablet	42 d	38
B2612A	Losartan 50 mg Tablet	43 d	27
B2612A	Warfarin 5 mg Tablet	44 d	46
B2612A	Cetirizine 10 mg Tablet	44 d	26
B2612A	Amiodarone 200 mg Tablet	45 d	45
B2612A	Pantoprazole 40 mg Tablet	46 d	34

Figure 21: Stock expiring within 60 days from v_near_expiry

Module 4: Audit and Reports

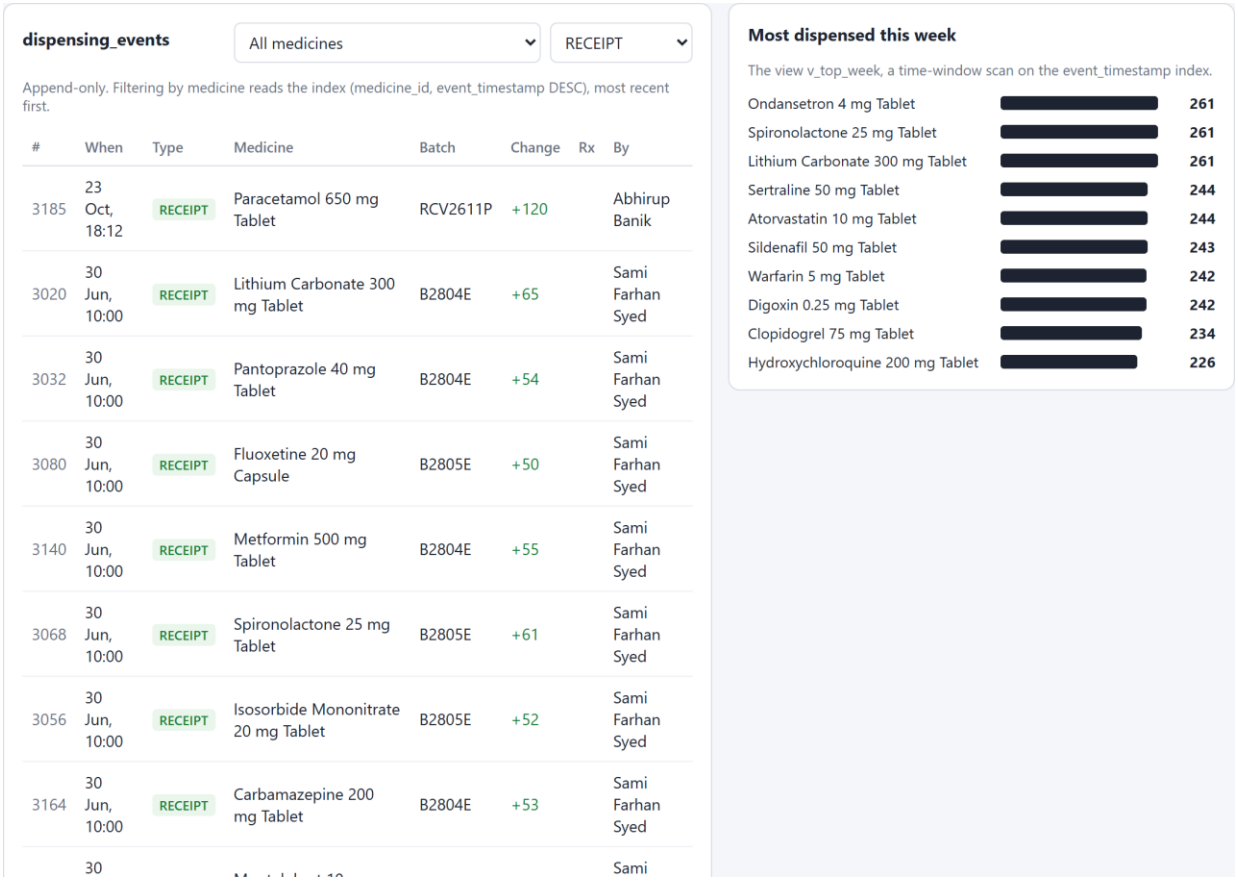


Figure 22: Audit log filtered to receipts, with the most dispensed medicines this week

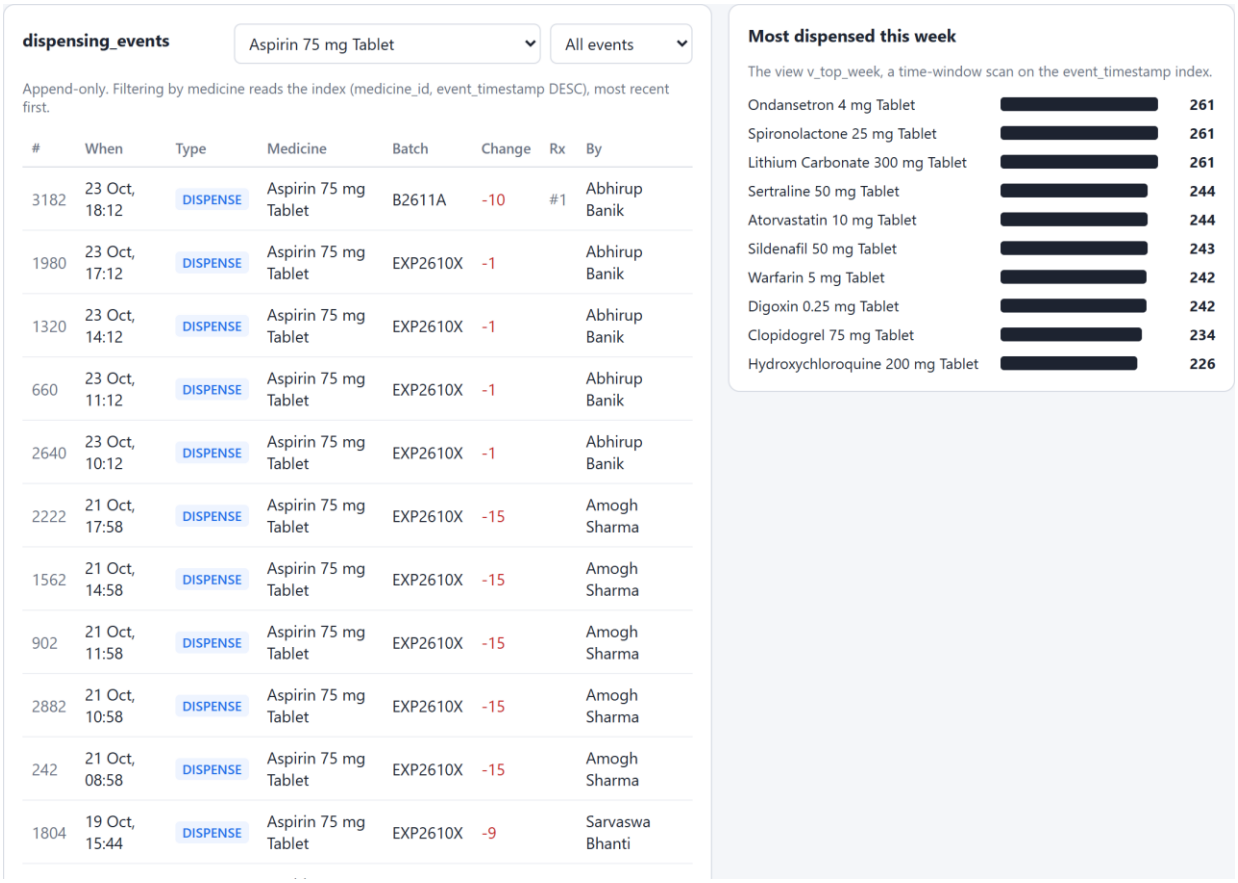


Figure 23: History of one medicine, newest first

Module 5: Interaction Knowledge

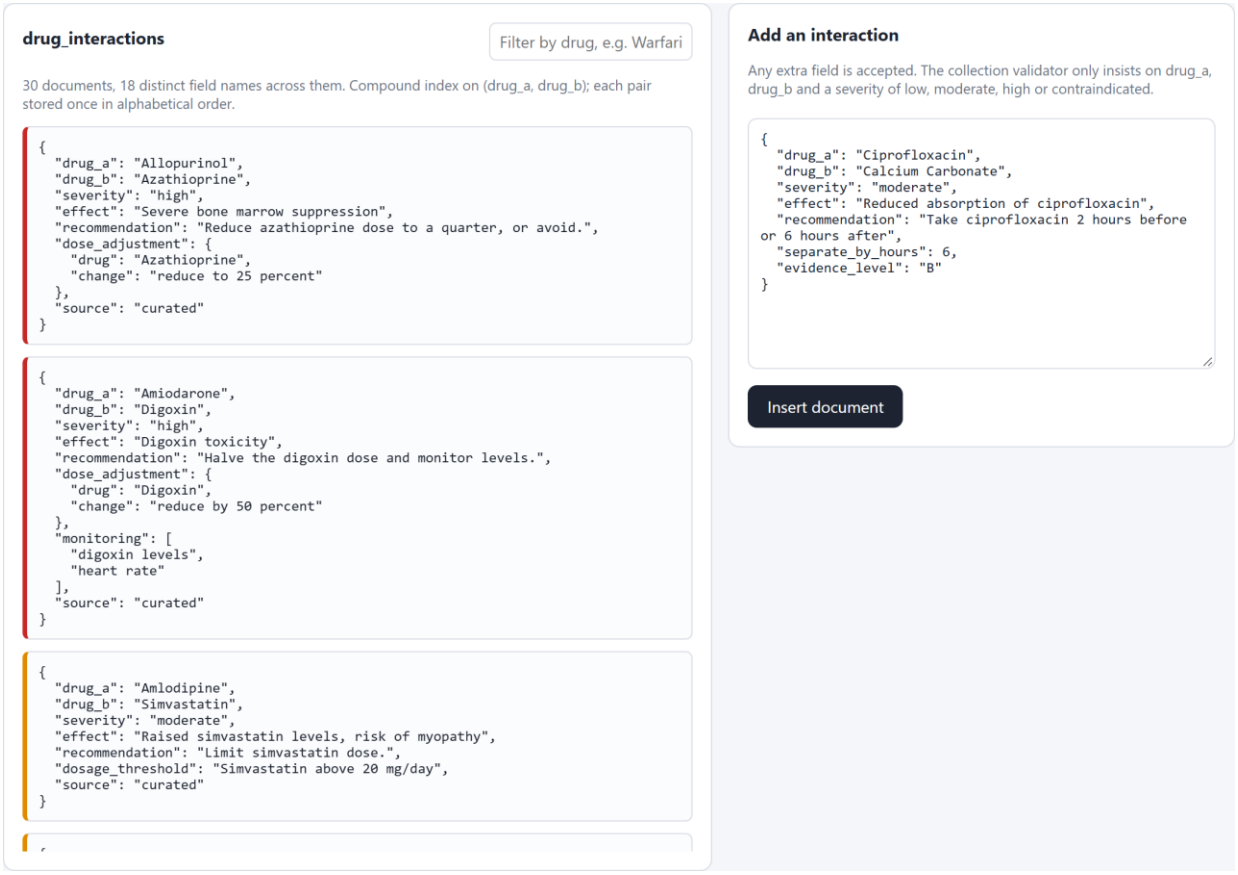


Figure 24: The drug_interactions collection; fields vary by pair

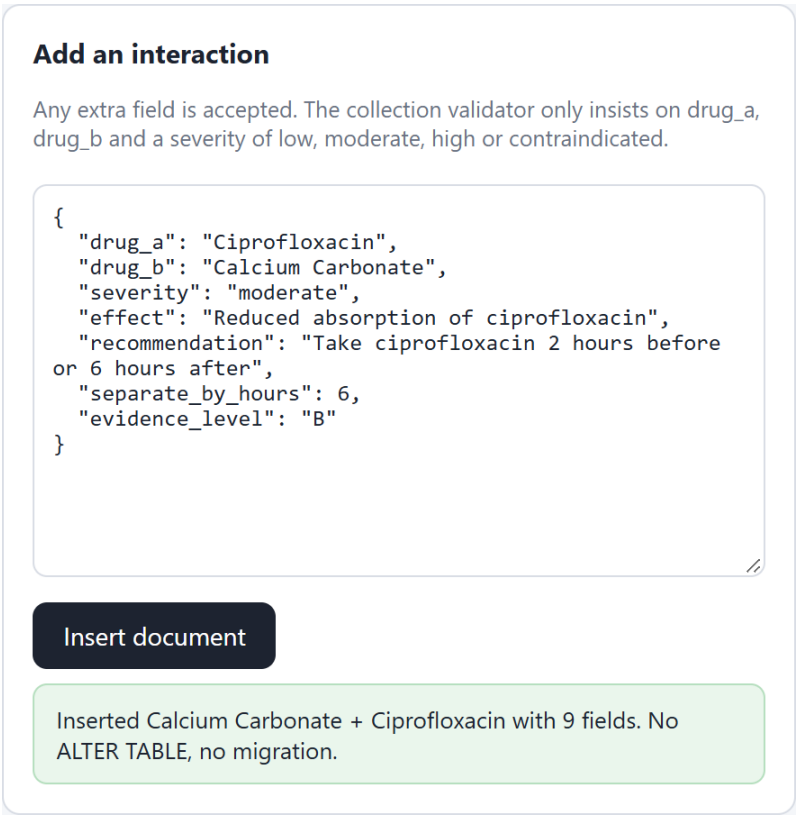


Figure 25: A document with new fields is accepted without any migration

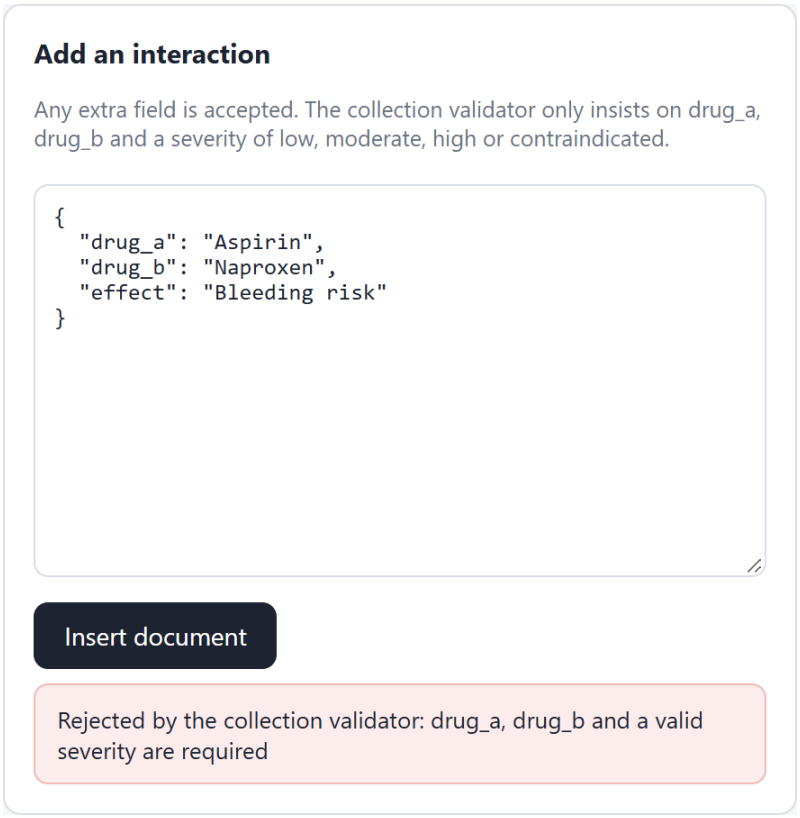


Figure 26: The $jsonSchema validator rejects a document without a severity

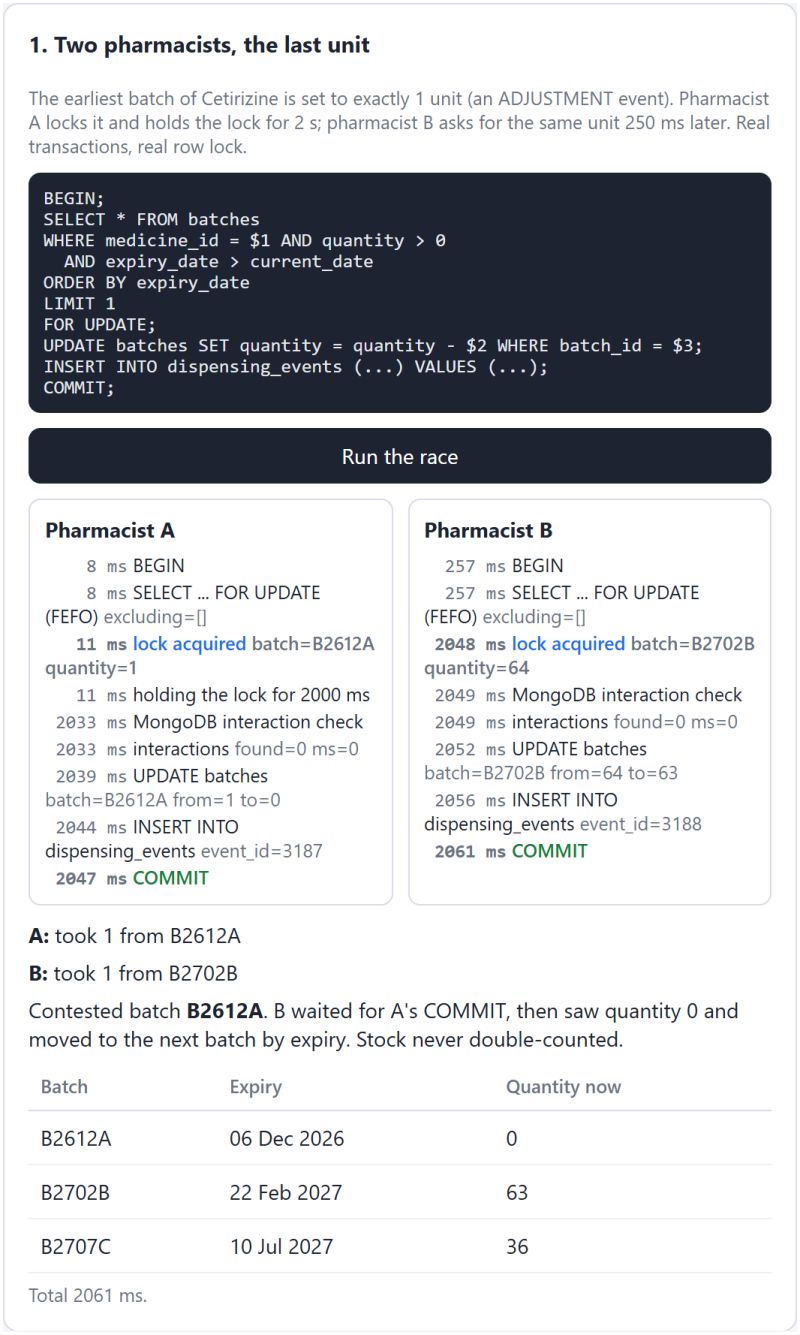


Figure 27: Two pharmacists, the last unit: the row lock serialises them

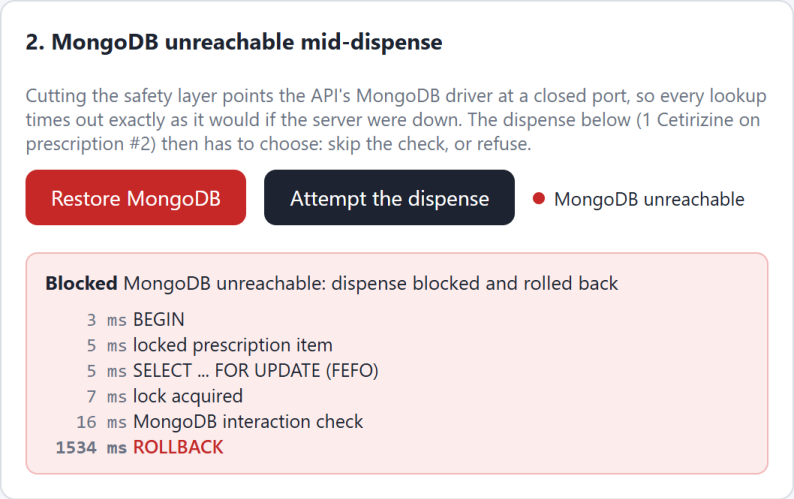


Figure 28: MongoDB unreachable: the dispense is blocked and rolled back

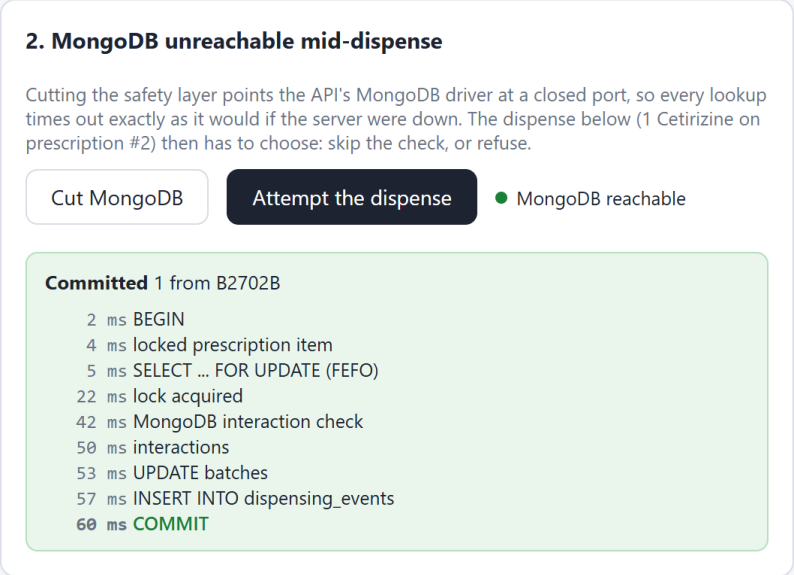


Figure 29: MongoDB restored: the same dispense commits



Figure 30: EXPLAIN ANALYZE: index scan against sequential scan

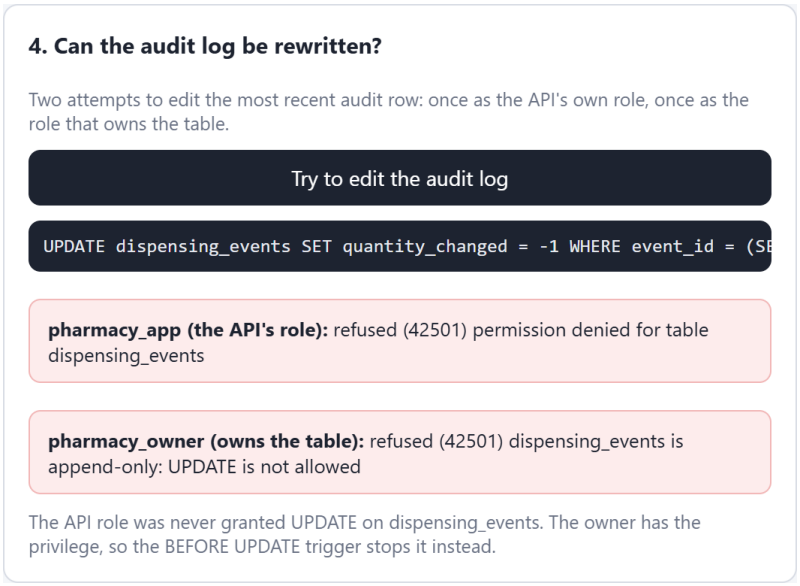


Figure 31: Both attempts to edit the audit log are refused

5.2 Source Code

The complete source code is in the project repository (https://github.com/Awwbhirup/DBMS_LAB_PROJECT). The listings below are the database layer in full and the dispense transaction, which together hold every rule described in this report.

Listing 1: Tables, keys and constraints `(sql/01_schema.sql)`

```
CREATE TABLE users (
    user_id      SERIAL PRIMARY KEY,
    name         VARCHAR(80)  NOT NULL,
    role         VARCHAR(20)  NOT NULL CHECK (role IN ('pharmacist', 'admin'))
);

CREATE TABLE medicines (
    medicine_id  SERIAL PRIMARY KEY,
    name         VARCHAR(120) NOT NULL UNIQUE,
    generic_name VARCHAR(80)  NOT NULL,
    manufacturer VARCHAR(80)  NOT NULL,
    form         VARCHAR(30)  NOT NULL,
    strength     VARCHAR(30)  NOT NULL,
    pack         VARCHAR(40)  NOT NULL
);

CREATE TABLE batches (
    batch_id      SERIAL PRIMARY KEY,
    medicine_id   INT          NOT NULL REFERENCES medicines(medicine_id) ON DELETE
```

```
        RESTRICT,
    batch_number  VARCHAR(30)  NOT NULL,
    expiry_date   DATE         NOT NULL,
    quantity      INT          NOT NULL CHECK (quantity >= 0),
    received_date DATE         NOT NULL DEFAULT current_date,
    UNIQUE (medicine_id, batch_number),
    CHECK (expiry_date > received_date)
);

CREATE TABLE prescriptions (
    prescription_id SERIAL PRIMARY KEY,
    patient_name    VARCHAR(80) NOT NULL CHECK (length(trim(patient_name)) > 0),
    patient_age     INT         CHECK (patient_age BETWEEN 0 AND 120),
    doctor_name     VARCHAR(80) NOT NULL CHECK (length(trim(doctor_name)) > 0),
    issued_date     DATE        NOT NULL DEFAULT current_date
);

CREATE TABLE prescription_items (
    item_id             SERIAL PRIMARY KEY,
    prescription_id     INT NOT NULL REFERENCES prescriptions(prescription_id) ON DELETE
        RESTRICT,
    medicine_id         INT NOT NULL REFERENCES medicines(medicine_id) ON DELETE RESTRICT,
    quantity_prescribed INT NOT NULL CHECK (quantity_prescribed > 0),
    UNIQUE (prescription_id, medicine_id)
);

CREATE TABLE dispensing_events (
    event_id         BIGSERIAL   PRIMARY KEY,
    medicine_id      INT         NOT NULL REFERENCES medicines(medicine_id),
    batch_id         INT         NOT NULL REFERENCES batches(batch_id),
    prescription_id  INT         REFERENCES prescriptions(prescription_id),
    event_type       VARCHAR(30) NOT NULL CHECK (event_type IN ('DISPENSE', 'RECEIPT',
        'ADJUSTMENT')),
    quantity_changed INT         NOT NULL CHECK (quantity_changed <> 0),
    performed_by     INT         NOT NULL REFERENCES users(user_id),
    event_timestamp  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (event_type <> 'DISPENSE' OR quantity_changed < 0),
    CHECK (event_type <> 'RECEIPT' OR quantity_changed > 0)
);
```

Listing 2: Indexes `(sql/02_indexes.sql)`

```
CREATE INDEX idx_batches_fefo
    ON batches (medicine_id, expiry_date)
    WHERE quantity > 0;


CREATE INDEX idx_events_medicine_time
    ON dispensing_events (medicine_id, event_timestamp DESC);


CREATE INDEX idx_events_time
    ON dispensing_events (event_timestamp);


CREATE INDEX idx_items_prescription
    ON prescription_items (prescription_id);


CREATE INDEX idx_medicines_name_lower
```

```
    ON medicines (lower(name) text_pattern_ops);

CREATE INDEX idx_medicines_generic_lower
    ON medicines (lower(generic_name) text_pattern_ops);
```

Listing 3: Append-only trigger on the audit log `(sql/03_audit_guard.sql)`

```
CREATE OR REPLACE FUNCTION reject_audit_change() RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION 'dispensing_events is append-only: % is not allowed', TG_OP
        USING ERRCODE = 'insufficient_privilege';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_events_no_update
    BEFORE UPDATE OR DELETE ON dispensing_events
    FOR EACH ROW EXECUTE FUNCTION reject_audit_change();

CREATE TRIGGER trg_events_no_truncate
    BEFORE TRUNCATE ON dispensing_events
    FOR EACH STATEMENT EXECUTE FUNCTION reject_audit_change();
```

Listing 4: Views `(sql/04_views.sql)`

```
CREATE VIEW v_stock AS
SELECT m.medicine_id,
       m.name,
       m.generic_name,
       COALESCE(SUM(b.quantity) FILTER (WHERE b.expiry_date > current_date), 0) AS
           usable_stock,
       COALESCE(SUM(b.quantity) FILTER (WHERE b.expiry_date <= current_date), 0) AS
           expired_stock,
       MIN(b.expiry_date) FILTER (WHERE b.quantity > 0 AND b.expiry_date > current_date) AS
           next_expiry,
       COUNT(b.batch_id) FILTER (WHERE b.quantity > 0) AS live_batches
FROM medicines m
LEFT JOIN batches b ON b.medicine_id = m.medicine_id
GROUP BY m.medicine_id, m.name, m.generic_name;

CREATE VIEW v_near_expiry AS
SELECT b.batch_id,
       b.batch_number,
       m.name,
       b.expiry_date,
       b.quantity,
       b.expiry_date - current_date AS days_left
FROM batches b
JOIN medicines m ON m.medicine_id = b.medicine_id
WHERE b.quantity > 0
  AND b.expiry_date <= current_date + 60
ORDER BY b.expiry_date;

CREATE VIEW v_top_week AS
SELECT m.medicine_id,
       m.name,
```

```
       -SUM(e.quantity_changed) AS units_dispensed,
       COUNT(*) AS dispenses
FROM dispensing_events e
JOIN medicines m ON m.medicine_id = e.medicine_id
WHERE e.event_type = 'DISPENSE'
  AND e.event_timestamp >= now() - interval '7 days'
GROUP BY m.medicine_id, m.name
ORDER BY units_dispensed DESC;


CREATE VIEW v_prescription_progress AS
SELECT pi.prescription_id,
       pi.medicine_id,
       pi.quantity_prescribed,
       COALESCE(-SUM(e.quantity_changed) FILTER (WHERE e.event_type = 'DISPENSE'), 0) AS
           quantity_dispensed
FROM prescription_items pi
LEFT JOIN dispensing_events e
       ON e.prescription_id = pi.prescription_id
      AND e.medicine_id = pi.medicine_id
GROUP BY pi.prescription_id, pi.medicine_id, pi.quantity_prescribed;
```

Listing 5: The two-phase dispense: preview and the confirm transaction (server/dispense.js)

```
import { pool, findInteractions, SafetyUnavailable } from "./db.js";


export class DispenseError extends Error {
  constructor(status, code, message, extra = {}) {
    super(message);
    this.status = status;
    this.code = code;
    this.extra = extra;
  }
}


const FEFO_SQL = `SELECT batch_id, batch_number, expiry_date, quantity
FROM batches
WHERE medicine_id = $1 AND quantity > 0 AND expiry_date > current_date
  AND batch_id <> ALL($2::int[])
ORDER BY expiry_date, batch_id
LIMIT 1
FOR UPDATE`;


export function fefoPlan(batches, quantity) {
  const plan = [];
  let left = quantity;
  for (const b of batches) {
    if (left <= 0) break;
    if (b.quantity <= 0 || b.expired) continue;
    const take = Math.min(b.quantity, left);
    plan.push({ batch_id: b.batch_id, batch_number: b.batch_number, expiry_date:
        b.expiry_date, take });
    left -= take;
  }
  return { plan, short_by: Math.max(left, 0) };
}
```

```
async function prescriptionContext(db, prescriptionId, medicineId) {
  const rx = await db.query(
    `SELECT pi.medicine_id, m.name, m.generic_name, p.quantity_prescribed,
       p.quantity_dispensed
     FROM prescription_items pi
     JOIN medicines m ON m.medicine_id = pi.medicine_id
     JOIN v_prescription_progress p ON p.prescription_id = pi.prescription_id AND
        p.medicine_id = pi.medicine_id
     WHERE pi.prescription_id = $1
     ORDER BY m.name`,
    [prescriptionId]
  );
  if (!rx.rowCount) throw new DispenseError(404, "no_prescription", "Prescription not
      found");
  const item = rx.rows.find((r) => r.medicine_id === medicineId);
  if (!item) throw new DispenseError(422, "not_on_prescription", "This medicine is not on
      the prescription");
  return { items: rx.rows, item };
}

function classify(found, generic) {
  const order = { contraindicated: 3, high: 2, moderate: 1, low: 0 };
  const list = found
    .map((d) => ({ ...d, _id: String(d._id), involves_this: d.drug_a === generic ||
        d.drug_b === generic }))
    .sort((a, b) => b.involves_this - a.involves_this || order[b.severity] -
        order[a.severity]);
  const blocking = list.filter((d) => d.involves_this && d.severity === "contraindicated");
  const needsAck = list.filter((d) => d.involves_this && d.severity !== "contraindicated"
      && d.severity !== "low");
  return { list, blocking, needsAck };
}

export async function preview({ prescription_id, medicine_id, quantity }) {
  const med = await pool.query("SELECT * FROM medicines WHERE medicine_id = $1",
      [medicine_id]);
  if (!med.rowCount) throw new DispenseError(404, "no_medicine", "Medicine not found");
  const medicine = med.rows[0];
  const batches = await pool.query(
    `SELECT batch_id, batch_number, expiry_date, quantity, received_date,
            expiry_date <= current_date AS expired,
            expiry_date - current_date AS days_left
     FROM batches
     WHERE medicine_id = $1 AND quantity > 0
     ORDER BY expiry_date, batch_id`,
    [medicine_id]
  );
  const { plan, short_by } = fefoPlan(batches.rows, quantity);
  let rx = null;
  let generics = [medicine.generic_name];
  if (prescription_id) {
    rx = await prescriptionContext(pool, prescription_id, medicine_id);
    generics = rx.items.map((i) => i.generic_name);
  }
  let safety;
  try {
    const r = await findInteractions(generics);
    const c = classify(r.found, medicine.generic_name);
    safety = { status: "ok", pairs_checked: r.pairs_checked, ms: r.ms, interactions:
        c.list, blocking: c.blocking.length > 0, needs_ack: c.needsAck.map((d) => d._id) };
  } catch (e) {
```

```
    if (!(e instanceof SafetyUnavailable)) throw e;
    safety = { status: "unavailable", message: e.message };
  }
  return {
    medicine,
    batches: batches.rows,
    plan,
    short_by,
    prescription: rx && { items: rx.items, remaining: rx.item.quantity_prescribed -
        rx.item.quantity_dispensed },
    safety
  };
}

export async function confirm(input, opts = {}) {
  const { prescription_id, medicine_id, quantity, user_id, acknowledged = [] } = input;
  const trace = opts.trace || (() => {});
  const holdMs = opts.holdMs || 0;
  if (!Number.isInteger(quantity) || quantity <= 0) throw new DispenseError(422,
      "bad_quantity", "Quantity must be a positive whole number");
  const client = await pool.connect();
  const taken = [];
  try {
    await client.query("BEGIN");
    trace("BEGIN");
    if (prescription_id) {
      await client.query(
        "SELECT item_id FROM prescription_items WHERE prescription_id = $1 AND medicine_id
            = $2 FOR UPDATE",
        [prescription_id, medicine_id]
      );
      trace("locked prescription item");
    }
    let left = quantity;
    const seen = [];
    while (left > 0) {
      trace("SELECT ... FOR UPDATE (FEFO)", { excluding: [...seen] });
      const r = await client.query(FEFO_SQL, [medicine_id, seen]);
      if (!r.rowCount) {
        throw new DispenseError(409, "insufficient_stock", `Not enough unexpired stock:
            short by ${left}`, { short_by: left });
      }
      const b = r.rows[0];
      trace("lock acquired", { batch: b.batch_number, quantity: b.quantity });
      const take = Math.min(b.quantity, left);
      taken.push({ batch_id: b.batch_id, batch_number: b.batch_number, expiry_date:
          b.expiry_date, before: b.quantity, take });
      seen.push(b.batch_id);
      left -= take;
    }
    if (holdMs) {
      trace(`holding the lock for ${holdMs} ms`);
      await new Promise((res) => setTimeout(res, holdMs));
    }
    const med = await client.query("SELECT generic_name FROM medicines WHERE medicine_id =
        $1", [medicine_id]);
    const generic = med.rows[0].generic_name;
    let generics = [generic];
    if (prescription_id) {
      const rx = await prescriptionContext(client, prescription_id, medicine_id);
      const remaining = rx.item.quantity_prescribed - rx.item.quantity_dispensed;
```

```
      if (quantity > remaining) {
        throw new DispenseError(409, "over_prescription", `Only ${remaining} left to
            dispense on this prescription`, { remaining });
      }
      generics = rx.items.map((i) => i.generic_name);
    }
    trace("MongoDB interaction check");
    let check;
    try {
      check = await findInteractions(generics);
    } catch (e) {
      if (e instanceof SafetyUnavailable) {
        throw new DispenseError(503, "safety_unavailable", "MongoDB unreachable: dispense
            blocked and rolled back");
      }
      throw e;
    }
    const c = classify(check.found, generic);
    trace("interactions", { found: check.found.length, ms: check.ms });
    if (c.blocking.length) {
      throw new DispenseError(409, "contraindicated", "Contraindicated combination:
          dispense refused", { interactions: c.blocking });
    }
    const missing = c.needsAck.filter((d) => !acknowledged.includes(d._id));
    if (missing.length) {
      throw new DispenseError(409, "needs_acknowledgement", "Interaction warning must be
          acknowledged", { interactions: missing });
    }
    const events = [];
    for (const t of taken) {
      await client.query("UPDATE batches SET quantity = quantity - $1 WHERE batch_id = $2",
          [t.take, t.batch_id]);
      trace("UPDATE batches", { batch: t.batch_number, from: t.before, to: t.before -
          t.take });
      const ev = await client.query(
        `INSERT INTO dispensing_events (medicine_id, batch_id, prescription_id, event_type,
            quantity_changed, performed_by)
         VALUES ($1, $2, $3, 'DISPENSE', $4, $5)
         RETURNING event_id, event_timestamp`,
        [medicine_id, t.batch_id, prescription_id || null, -t.take, user_id]
      );
      events.push(ev.rows[0]);
      trace("INSERT INTO dispensing_events", { event_id: ev.rows[0].event_id });
    }
    await client.query("COMMIT");
    trace("COMMIT");
    return { ok: true, taken, events, interactions: c.list };
  } catch (e) {
    await client.query("ROLLBACK").catch(() => {});
    trace("ROLLBACK", { reason: e.code || e.message });
    throw e;
  } finally {
    client.release();
  }
}
```

Listing 6: Connection pools, MongoDB client and the interaction lookup (server/db.js)

```
import "dotenv/config";
import pg from "pg";
```

```
import { MongoClient } from "mongodb";

const env = process.env;

pg.types.setTypeParser(1082, (v) => v);
pg.types.setTypeParser(20, (v) => Number(v));

export const pool = new pg.Pool({
  host: env.PGHOST,
  port: Number(env.PGPORT),
  database: env.PGDATABASE,
  user: env.PGUSER,
  password: env.PGPASSWORD,
  max: 12
});

export const ownerPool = new pg.Pool({
  host: env.PGHOST,
  port: Number(env.PGPORT),
  database: env.PGDATABASE,
  user: env.PGOWNER_USER,
  password: env.PGOWNER_PASSWORD,
  max: 2
});

for (const p of [pool, ownerPool]) p.on("error", () => {});

const DEAD_URL = "mongodb://127.0.0.1:1";
let mongo = null;
let outage = false;

export function mongoOutage() {
  return outage;
}

export async function setMongoOutage(on) {
  outage = on;
  if (mongo) {
    await mongo.close().catch(() => {});
    mongo = null;
  }
}

export async function interactionsCollection() {
  if (!mongo) {
    mongo = new MongoClient(outage ? DEAD_URL : env.MONGO_URL, {
      serverSelectionTimeoutMS: 1500,
      connectTimeoutMS: 1500
    });
    try {
      await mongo.connect();
    } catch (e) {
      mongo = null;
      throw e;
    }
  }
  return mongo.db(env.MONGO_DB).collection("drug_interactions");
```

```
}

export class SafetyUnavailable extends Error {
  constructor(cause) {
    super("Interaction check could not be completed: MongoDB unreachable");
    this.cause = cause;
  }
}

export async function findInteractions(genericNames) {
  const names = [...new Set(genericNames)].sort((a, b) => a.localeCompare(b));
  const pairs = [];
  for (let i = 0; i < names.length; i++) {
    for (let j = i + 1; j < names.length; j++) pairs.push({ drug_a: names[i], drug_b:
        names[j] });
  }
  if (!pairs.length) return { pairs_checked: 0, found: [], ms: 0 };
  const t = performance.now();
  let col;
  try {
    col = await interactionsCollection();
    const found = await col.find({ $or: pairs }, { projection: { added_at: 0 } }).toArray();
    return { pairs_checked: pairs.length, found, ms: Math.round((performance.now() - t) *
        10) / 10 };
  } catch (e) {
    await setMongoOutage(outage);
    throw new SafetyUnavailable(e);
  }
}
```

Listing 7: Database setup: roles, privileges, MongoDB validator and index (scripts/setup.js)

```
import "dotenv/config";
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import pg from "pg";
import { MongoClient } from "mongodb";
import interactions from "../mongo/interactions.js";

const root = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const env = process.env;
const ident = (s) => '"' + s.replace(/"/g, '""') + '"';
const lit = (s) => "'" + s.replace(/'/g, "''") + "'";

function log(msg) {
  console.log(new Date().toISOString().slice(11, 19) + "  " + msg);
}

async function connect(database, user, password) {
  const c = new pg.Client({ host: env.PGHOST, port: Number(env.PGPORT), database, user,
      password });
  await c.connect();
  return c;
}
```

```
async function setupPostgres() {
  const su = await connect("postgres", env.PG_SUPERUSER, env.PG_SUPERPASS);
  for (const [role, pw] of [[env.PGOWNER_USER, env.PGOWNER_PASSWORD], [env.PGUSER,
      env.PGPASSWORD]]) {
    const exists = await su.query("SELECT 1 FROM pg_roles WHERE rolname = $1", [role]);
    const verb = exists.rowCount ? "ALTER" : "CREATE";
    await su.query(`${verb} ROLE ${ident(role)} LOGIN PASSWORD ${lit(pw)}`);
  }
  await su.query(`DROP DATABASE IF EXISTS ${ident(env.PGDATABASE)} WITH (FORCE)`);
  await su.query(`CREATE DATABASE ${ident(env.PGDATABASE)} OWNER
      ${ident(env.PGOWNER_USER)}`);
  await su.end();
  log("postgres: database " + env.PGDATABASE + " created");

  const owner = await connect(env.PGDATABASE, env.PGOWNER_USER, env.PGOWNER_PASSWORD);
  await owner.query("REVOKE ALL ON SCHEMA public FROM PUBLIC");
  const files = fs.readdirSync(path.join(root, "sql")).filter((f) =>
      f.endsWith(".sql")).sort();
  for (const f of files) {
    const t = Date.now();
    await owner.query(fs.readFileSync(path.join(root, "sql", f), "utf8"));
    log(`postgres: ${f} (${Date.now() - t} ms)`);
  }
  const app = ident(env.PGUSER);
  await owner.query(`GRANT USAGE ON SCHEMA public TO ${app}`);
  await owner.query(`GRANT SELECT ON ALL TABLES IN SCHEMA public TO ${app}`);
  await owner.query(`GRANT INSERT, UPDATE ON batches, prescriptions, prescription_items TO
      ${app}`);
  await owner.query(`GRANT INSERT ON dispensing_events TO ${app}`);
  await owner.query(`GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO ${app}`);
  const counts = await owner.query(`
    SELECT (SELECT count(*) FROM medicines) AS medicines,
           (SELECT count(*) FROM batches WHERE quantity > 0) AS live_batches,
           (SELECT count(*) FROM batches) AS all_batches,
           (SELECT count(*) FROM prescriptions) AS prescriptions,
           (SELECT count(*) FROM dispensing_events) AS events`);
  log("postgres: " + JSON.stringify(counts.rows[0]));
  await owner.end();
}

function normalise(doc) {
  const [a, b] = [doc.drug_a, doc.drug_b].sort((x, y) => x.localeCompare(y));
  return { ...doc, drug_a: a, drug_b: b };
}

async function setupMongo() {
  const client = new MongoClient(env.MONGO_URL, { serverSelectionTimeoutMS: 4000 });
  await client.connect();
  const db = client.db(env.MONGO_DB);
  await db.dropDatabase();
  await db.createCollection("drug_interactions", {
    validator: {
      $jsonSchema: {
        bsonType: "object",
        required: ["drug_a", "drug_b", "severity"],
        properties: {
          drug_a: { bsonType: "string" },
          drug_b: { bsonType: "string" },
          severity: { enum: ["low", "moderate", "high", "contraindicated"] }
```

```
        }
      }
    }
  });
  const col = db.collection("drug_interactions");
  await col.createIndex({ drug_a: 1, drug_b: 1 }, { unique: true, name: "pair" });
  const docs = interactions.map(normalise).map((d) => ({ ...d, source: d.source ??
      "curated", added_at: new Date() }));
  await col.insertMany(docs);
  log(`mongo: ${docs.length} interaction documents, index on (drug_a, drug_b)`);
  await client.close();
}


const only = process.argv[2];
try {
  if (only !== "mongo") await setupPostgres();
  if (only !== "postgres") await setupMongo();
  log("done");
} catch (e) {
  console.error(e.message);
  process.exit(1);
}
```

6. CONCLUSION

The project shows that the two halves of a pharmacy's data are best served by two different databases, and that they can be combined without giving up correctness. PostgreSQL enforces every rule that must be exact: keys and CHECK constraints make invalid states impossible to store, a single short transaction with row locks makes each dispense atomic and safe under concurrency, a partial index keeps FEFO selection fast as history grows, and a trigger together with role privileges keeps the audit log append-only. MongoDB holds interaction knowledge whose shape varies from pair to pair, with a validator that sets a minimum standard, and the dispense refuses to proceed whenever that knowledge cannot be consulted.

Every one of these properties was demonstrated live and measured, and the automated checks confirm them on every run. Future work would add authenticated users with roles in the interface, reorder alerts based on stock levels, and loading interaction data from a published source in place of the curated seed.

7. REFERENCES

[1] P. J. Sadalage and M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Upper Saddle River, NJ, USA: Addison-Wesley, 2012.

[2] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.

[3] E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377-387, 1970.

[4] J. Gray and A. Reuter, Transaction Processing: Concepts and Techniques. San Mateo, CA, USA: Morgan Kaufmann, 1992.

[5] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil and P. O'Neil, "A critique of ANSI SQL isolation levels," in Proc. ACM SIGMOD Int. Conf. on Management of Data, 1995, pp. 1-10.

[6] The PostgreSQL Global Development Group, "PostgreSQL 17 Documentation, Chapter 13: Concurrency Control." https://www.postgresql.org/docs/17/mvcc.html

[7] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[8] The PostgreSQL Global Development Group, "PostgreSQL 17 Documentation, Section 11.8: Partial Indexes." https://www.postgresql.org/docs/17/indexes-partial.html

[9] M. Stonebraker and U. Cetintemel, "'One size fits all': an idea whose time has come and gone," in Proc. 21st Int. Conf. on Data Engineering (ICDE), 2005, pp. 2-11.

[10] R. Hecht and S. Jablonski, "NoSQL evaluation: a use case oriented survey," in Proc. Int. Conf. on Cloud and Service Computing, 2011, pp. 336-341.

[11] MongoDB, Inc., "MongoDB Manual: Schema Validation." https://www.mongodb.com/docs/manual/core/schema-validation/

[12] D. S. Wishart et al., "DrugBank 5.0: a major update to the DrugBank database for 2018," Nucleic Acids Research, vol. 46, no. D1, pp. D1074-D1082, 2018.

[13] World Health Organization, "Good storage and distribution practices for medical products," WHO Technical Report Series, no. 1025, Annex 7, 2020.